



Centro de Ciencias Básicas

Departamento de Ciencias de la Computación

Thesis:

Hybrid Log Parsing: Combining Semantic and Statistical Techniques for Robust Template Extraction and Parsing

Presented by:

Diego Emilio Moreno Sánchez

To obtain: El grado de Ingeniería en Computación Inteligente

Director:

Dr. Arturo Barajas Saavedra

Committee:

Dr. Francisco Javier Álvarez Rodríguez
Dr. Alejandro Padilla Díaz

Aguascalientes, Ags Mex
May 2026

Abstract

Modern software systems generate enormous volumes of log data that serve as a primary source of observability for debugging, monitoring, and incident response. In large-scale production environments, the manual analysis of these logs is neither practical nor sustainable, making automated log parsing a critical capability for Site Reliability Engineers and system administrators. Log parsing transforms raw, unstructured log lines into structured representations by identifying constant template tokens and variable parameter fields, enabling downstream automated analysis. Despite decades of research, existing parsing solutions force practitioners to choose between two incompatible strengths: statistical parsers, which group logs consistently across large corpora at high speed, and semantic parsers, which achieve precise per-line token extraction but lose global grouping coherence at scale. No single approach reliably provides both properties simultaneously, leaving a meaningful gap in the current state of the art.

This thesis addresses that gap through the design, implementation, and evaluation of DrainUP, a hybrid log parsing system that combines a statistical parser (Drain) with a semantic parser (UniParser) through a confidence-based routing mechanism. The core idea is that Drain processes every incoming log line and produces a numerical confidence score reflecting the quality of its template match. Lines that exceed a configurable threshold are accepted as parsed by Drain, while lines that fall below it are forwarded to UniParser for a semantic second pass. The outputs of both stages are then merged into a single structured result. This design aims to preserve Drain's speed and global grouping consistency for the majority of log lines while leveraging UniParser's semantic precision for the ambiguous or irregular cases that heuristic parsing cannot handle reliably.

The system was evaluated using the LogHub 2.0 benchmark across thirteen heterogeneous real-world log datasets under two configurations: a controlled 2,000-line sample per dataset and the full annotated log contents, which range from approximately 21,000 to over sixteen million lines per source. DrainUP was compared against unmodified Drain and unmodified UniParser as direct baselines. Results show that DrainUP outperforms both baselines on Grouping Accuracy (average 0.871, versus 0.844 for Drain and 0.674 for UniParser on the full dataset) and on the F1 of Group Accuracy (0.763, versus 0.578 for Drain and 0.509 for UniParser), while avoiding the catastrophic near-zero grouping failures that both baselines exhibit on specific log sources. The confidence-based routing mechanism adds an average of only 20.8 seconds to Drain's parse time across the full evaluation suite, a negligible overhead in any operational context. The findings confirm that the hybrid paradigm, combining complementary parsing strategies through a lightweight routing mechanism, represents a practical and robust direction for log parsing in heterogeneous production environments.

Table of Contents

1. Introduction.....	7
1.1. Importance for software engineering and incident management.....	7
1.2. Role of system specialists (SREs, application owners, sysadmins).....	7
1.3. Current practices and their limitations.....	7
1.4. Practical investigation workflow (example).....	8
1.5. Motivation.....	8
2. Problem Statement.....	8
3. Social and Ethical Justification.....	8
4. Investigation Objectives.....	9
5. Research Questions.....	9
5.1. RQ1 — Accuracy trade-off:.....	9
5.2. RQ2 — Workload distribution, cost & latency:.....	9
5.3. RQ3 — Deployment guidelines from benchmark results:.....	10
6. Theoretical Fundamentals and Related work.....	10
6.1. Definitions and Terminology.....	10
6.2. Anchor study: a large-scale evaluation benchmark.....	12
6.3. Taxonomy of parsing approaches and thesis plan.....	13
6.4. Frequency-based Parsing.....	13
6.4.1. Logram: Efficient Log Parsing Using n-Gram Dictionaries.....	13
6.4.2. LogCluster - A data clustering and pattern mining algorithm for event logs.....	14
6.5. Similarity-based Parsing.....	15
6.5.1. LogMine: Fast Pattern Recognition for Log Analytics.....	15
6.5.2. Length Matters: Clustering System Log Messages using Length of Words.....	17
6.6. Heuristics-based Parsing.....	18
6.6.1. Drain: An Online Log Parsing Approach with Fixed Depth Tree.....	18
6.6.2. Abstracting Execution Logs to Execution Events for Enterprise Applications (Short Paper).....	19
6.7. Semantic-based Parsing.....	21
6.7.1. SemParser: A Semantic Parser for Log Analytics.....	21
6.7.2. Log Parsing with Prompt-based Few-shot Learning.....	22
6.7.3. UniParser: A Unified Log Parser for Heterogeneous Log Data.....	23
6.8. General review of LLM approaches.....	24
6.9. Evaluation Criteria and Datasets Commonly Used.....	25
6.10. Open challenges on the current state of the art.....	27
7. Investigation Method.....	28
7.1. Research Design.....	28
7.2. Framework Pipeline Overview.....	28

8. Methodology and Implementation.....	29
8.1. Parser Selection for the Hybrid Approach.....	29
8.2. System Architecture Overview.....	31
8.3. Modified Drain: Confidence-Aware Parsing.....	32
8.4. Low-Confidence Routing and UniParser Integration.....	33
8.5. Merge Pipeline.....	33
9. Experimentation and Testing.....	34
9.1. Hardware Setup.....	34
9.2. Benchmark and Dataset Scope.....	35
9.3. Baseline Configuration.....	35
9.4. Evaluation Protocol.....	36
10. Results — Analysis and Interpretation.....	37
10.1. Experimental Setup Recap.....	37
10.2. Full Dataset Results.....	38
10.2.1. Grouping Accuracy (GA).....	38
10.2.2. F1 of Group Accuracy (FGA).....	40
10.2.3. Parsing Accuracy (PA).....	42
10.2.4. Speed at Scale.....	44
10.2.5. Overall Full Dataset Assessment.....	45
10.3. 2k Dataset Results.....	47
10.3.1. Grouping Accuracy (GA) and FGA.....	47
10.3.2. Parsing Accuracy (PA).....	49
10.3.3. Speed on 2k.....	50
10.3.4. Overall 2k Assessment.....	51
10.4. Cross-Dataset Analysis.....	53
10.5. Connecting Results to Research Questions.....	54
10.5.1. RQ1: Does DrainUP achieve superior accuracy relative to its standalone constituent parsers?.....	54
10.5.2. RQ2: How does the confidence-based routing mechanism affect the accuracy–speed trade-off?.....	55
10.5.3. RQ3: What deployment considerations and configuration guidelines can be derived from the benchmark results?.....	55
10.6. Deployment Guidelines and Practical Recommendations.....	56
11. Conclusions.....	57
12. References.....	59

Index of Tables

Table i: Table with the most important metrics used within the LogHub benchmark.....	28
Table ii: Efficiency metrics, such as time spent in average per dataset, a hint to its slowest dataset and if the given parser actually completed parsing all the datasets (given a 12 hours limit).....	28
Table iii: Hardware and software environment comparison between the original LogHub evaluation setup and the DrainUP experimental environment used in this thesis.....	33
Table iv: Overview of the thirteen evaluation datasets, showing the number of annotated log lines in the full dataset and the number of distinct ground-truth templates in both the full and 2k benchmark configurations. A dash (—) indicates that the 2k template count was not reported in the LogHub 2.0 documentation for that dataset.....	36
Table v: Per-dataset metric winners on the full dataset for GA, FGA, and PA. Green shading indicates DrainUP wins; amber shading indicates Drain wins; blue shading indicates UniParser wins; gray indicates a tie. The bottom row summarizes the total win counts per parser per metric.....	52

Table of Figures

Figure i: Equation for the Precision of Group Accuracy.....	25
Figure ii: Equation for Recall of Group Accuracy.....	25
Figure iii: Equation for the F1 of Group Accuracy.....	25
Figure iv: Equation for Precision of Template Accuracy.....	25
Figure v: Equation for Recall of Template Accuracy.....	25
Figure vi: Equation for F1 of Template Accuracy.....	26
Figure vii: High-level architecture Diagram of the pipeline used by DrainUP.....	30
Figure viii: Equation for Similarity Score equation used in Drain's code.....	31
Figure ix: Grouping Accuracy (GA) on the full dataset for all three parsers across the thirteen evaluation datasets.....	38
Figure x: F1 of Group Accuracy (FGA) on the full dataset.....	40
Figure xi: Parsing Accuracy (PA) on the full dataset.....	42
Figure xii: Parsing Times for the 3 parsers on the 13 datasets.....	43
Figure xiii: Average Performance for the 3 parsers across the full dataset.....	44
Figure xiv: Parser Performance radar chart for the 6 metrics and the 3 parsers for the full dataset.....	45
Figure xv: Speed vs Accuracy Trade-off for GA, PA and FGA.....	46
Figure xvi: Grouping Accuracy (GA) on the 2k dataset.....	47
Figure xvii: F1 of Grouping Accuracy (FGA) on the 2k dataset.....	48
Figure xviii: Parsing Accuracy (PA) on the 2k dataset.....	49
Figure xix: Parsing times on the 2k dataset.....	50
Figure xx: Average Performance for the 3 parsers across the 2k dataset.....	51
Figure xxi: Parser Performance radar chart for the 6 metrics and the 3 parsers for the 2k dataset.....	52

1. Introduction

Logs are a primary source of observability data in modern software systems, generated by applications, middleware, and infrastructure components. They provide chronological records of events, errors, and operational states that are essential for debugging, monitoring, and security analysis. However, raw logs are often verbose and unstructured, which makes automated analysis difficult. Template extraction, also referred to as log parsing, transforms raw log lines into a structured representation by identifying templates and the variable parameters they contain, enabling subsequent automated processing and analytics.

1.1. Importance for software engineering and incident management

In software engineering and incident management, logs play a fundamental role in triaging production problems. Logs frequently contain detailed information about system behavior, including transaction outcomes and component states. In large enterprise systems, the volume of generated logs can be enormous, and locating the few lines that actually explain a failure becomes a significant challenge. Abstracting log lines into execution events is difficult and time-consuming. A domain expert is often required to determine which log lines correspond to the same execution event, and these experts are rarely available and typically have limited time. Moreover, abstractions must be updated for each new version of the software system, which further increases maintenance effort. [1]

1.2. Role of system specialists (SREs, application owners, sysadmins)

System experts include application owners, system administrators, and Site Reliability Engineers (SREs). The SRE role was popularized roughly two decades ago at Google, and it combines software engineering and operations responsibilities to improve reliability and automate operational work [2]. In practice, the relationship between SREs and logs is tightly coupled: effective incident response depends on both high-quality observability data and the expert knowledge that these specialists can provide.

1.3. Current practices and their limitations

Today, root-cause analysis and failure remediation commonly rely on custom scripts that automate parts of log processing and analysis. These scripts are often implemented with regular expressions or ad hoc rules developed by individual engineers; they may be stored in large rule-based systems, such as log pipelines with Grok parsers in Datadog [3]. When automated rules fail to explain an issue, engineers revert to manual, time-consuming searches through thousands of log lines and source files. This approach has two main weaknesses: (1) rules and scripts become outdated as software and logs evolve, and (2) when experienced engineers leave, their domain knowledge is lost, reducing the organization's ability to respond efficiently to incidents [4].

1.4. Practical investigation workflow (example)

In professional practice, incidents are investigated using centralized log platforms. The typical workflow begins by reviewing the alerts that triggered when the incident began; relevant logs are then inspected to identify symptoms and narrow potential root causes. Consider a container in Kubernetes that has restarted: the restart could be caused by an application crash, an out-of-memory (OOM) condition, or a container runtime issue. By querying the logging system, for example Sumo Logic, and correlating metrics such as memory usage with error messages, indicators such as Java OOM errors can be searched for. Although garbage-collected languages avoid certain classes of memory bugs, they can still exhibit memory leaks caused by retained references; severe leaks can exhaust memory and crash programs [5], [6].

1.5. Motivation

As illustrated above, log analysis for production troubleshooting and post-mortem investigations requires significant human effort: knowledge of the system, skill with logging tools, and the ability to correlate multiple signals to reach a conclusion. Automating template extraction and robust parsing would reduce dependence on individual experts, improve the speed of incident triage, and help retain organizational knowledge despite staff turnover.

2. Problem Statement

Existing log parsing solutions trade off between two incompatible strengths: semantic (local) parsers, which achieve high accuracy in extracting fine-grained event structure from individual log lines, and statistical (global) parsers, which group logs consistently across a corpus using pattern statistics. No single approach reliably provides both precise local parsing and coherent global grouping. As a result, practitioners must choose between per-log correctness and overall consistency, which reduces parser utility in real SRE environments where logs come from diverse frameworks, heterogeneous systems, and rapidly evolving formats. The core deficiency is the lack of a unified method that integrates semantic understanding for local accuracy with statistical patterning for global consistency, leaving many parsers context-dependent and unable to generalize across systems and logging practices.

3. Social and Ethical Justification

This research aims to improve the reliability and usefulness of log-parsing tools for SREs, incident responders, and security analysts, with a practical benefit and clear social value: faster and more accurate incident detection and diagnosis can reduce system downtime, lessen the harms caused by service outages, and lower the manual burden of triage. However, working with operational logs raises non-trivial ethical and social concerns, including privacy, security, fairness, environmental cost, and the potential for misuse.

This work focuses on improving parsing accuracy and template stability; it does not attempt to make downstream legal or organizational decisions automatically. This thesis will use only open-source datasets that do not contain personal or sensitive information.

4. Investigation Objectives

This thesis has three main objectives. The first objective is to design and implement a hybrid log parsing architecture that effectively combines semantic-based parsers for local, token-level analysis with statistical-based parsers for global pattern recognition and grouping. This objective involves producing a modular, end-to-end pipeline in which a semantic component is responsible for extracting tokens, fields, and intent from individual log lines, while a statistical component performs template discovery, clustering, and refinement. The architecture will include a routing or selector mechanism that determines, based on confidence or rarity, which lines are escalated to the semantic module and which are handled by the statistical module.

The second objective is to conduct a comprehensive evaluation comparing the hybrid parser’s accuracy and computational efficiency against state-of-the-art semantic-based and statistical-based baseline parsers using publicly available benchmark datasets, specifically LogHub 2.0. The evaluation will use the metrics provided by the LogHub benchmark to compare the proposed solution with the baseline parsers.

The third objective is to analyze the benchmark evaluation results in order to derive practical deployment guidelines and configuration recommendations for integrating the hybrid log parsing approach into heterogeneous production environments. The goal is not to perform a live production deployment, but rather to translate the empirical findings from the benchmark into actionable recommendations for SRE teams considering adoption.

5. Research Questions

5.1. RQ1 — Accuracy trade-off:

Question: Does a hybrid parser that combines semantic and statistical methods achieve superior parsing accuracy compared with standalone semantic and standalone statistical baselines across diverse log formats?

Why it matters (motivation): Semantic parsers excel at per-line field extraction, while statistical parsers excel at corpus-level template consistency. Demonstrating that a hybrid approach outperforms both baselines would support the core claim of the thesis.

Hypothesis: A hybrid semantic-plus-statistical parser will achieve higher per-line extraction performance and better template grouping scores than either a semantic-only parser or a statistical-only parser, on average across heterogeneous datasets.

5.2. RQ2 — Workload distribution, cost & latency:

Question: How does the distribution of parsing workload between semantic and statistical components affect the trade-off among parsing accuracy, computational cost, and processing latency?

Why it matters (motivation): Real deployments are constrained by cost and latency. Semantic-based parsing is often more expensive and slower than statistical methods, so a hybrid approach must support controllable trade-offs.

Hypothesis: Carefully allocating high-value or ambiguous lines to the semantic component while routing routine lines to the statistical component will yield near-semantic accuracy at significantly lower cost and latency than a semantic-only pipeline.

5.3. RQ3 — Deployment guidelines from benchmark results:

Question: What deployment considerations and configuration guidelines can be derived from benchmark evaluation results to inform the integration of the hybrid parsing approach into real-world SRE production environments?

Why it matters (motivation): Benchmark results alone are insufficient for practitioners making adoption decisions. Concrete guidance on configuration, trade-offs, and operational fit is needed to bridge the gap between controlled evaluation and real-world applicability.

Hypothesis: Benchmark evaluation results across heterogeneous datasets will reveal consistent patterns in confidence-threshold behavior, workload distribution, and accuracy-latency trade-offs that can be systematically translated into deployment guidelines applicable to diverse SRE production environments.

6. Theoretical Fundamentals and Related work

Having motivated the study and established common terminology, the next step is to review the current state of the art in template extraction and log parsing. A thorough literature review will: (1) situate this thesis within the existing body of work, (2) identify representative approaches and baselines for comparison, and (3) reveal open problems that justify the proposed contributions.

6.1. Definitions and Terminology

To avoid ambiguity, this section defines the key terms used throughout this work.

Log. Log data consists of recorded information generated by logging facilities in software applications, systems, devices, or networks during operation [7].

Log Parsing. log parsing extracts the constant parts (i.e., log templates) and the changeable parts (i.e., log parameters) from log messages. In the above example, the log template is ‘connected to host: <*>’, and the log parameter indicates the concrete IP address of the host, e.g., ‘172.16.254.1’. [8].

Log Level. Logs are produced by logging statements that developers insert into the source code, with each logging statement a log level is specified (e.g., debug, info, warn, error, fatal). This mechanism allows developers and users to suppress logs for less critical events to focus on error or critical logs. [9].

Structured Log. A structured log is a log with a defined, consistent schema or typed fields that downstream systems can reliably parse and interpret. The textual encoding can be JSON, protobuf, or another format, but what makes a log structured is the presence of a stable schema (field names, types, and semantics), not merely that it is valid JSON. [10]

Unstructured Log. Unstructured logs are logs that don't follow a consistent structure. They may be more human-readable, and are often used in development. However, it is not preferred to use unstructured logs for production observability purposes, since they are much more difficult to parse and analyze at scale. [10].

Log Stream. A log stream is an application specific collection of data that is used as a log. The data is written to and read from the log stream by one or more instances of the application associated with the log stream. A log stream can be used for such purposes as a transaction log, a log for re-creating databases, a recovery log, or other logs needed by applications. [11]

Log Template. A log template is a predefined format used to log system events in software applications. It typically includes placeholders for essential details like timestamps, log levels (e.g., INFO, ERROR), message content, contextual metadata and more [12].

Tokenisation. In the area of log parsing, Tokenisation involves splitting log entries into smaller units (tokens) based on predefined delimiters (e.g., spaces, commas). Each token is then assigned to a specific field. [13]

Caching. In computing, a cache is a high-speed data storage layer which stores a subset of data, typically transient in nature, so that future requests for that data are served up faster than is possible by accessing the data's primary storage location. [14] And the same concept can be applied to logs as a form of data.

Throughput. Throughput refers to the average volume of data that can actually pass through the network over a specific time. It indicates the number of data packets that arrive at their destinations successfully and the data packet loss. [15]

Latency. Latency is the delay in network communication. It shows the time that data takes to transfer across the network. Networks with a longer delay or lag have high latency, while those with fast response times have lower latency. [15]

Hashing. The process of using a mathematical algorithm against data to produce a numeric value that is representative of that data. [16]

Encoder. Use a system of symbols to represent information, which might originally have some other representation [17]

Online and Offline Log Parsers. The log parsing approaches can be categorized into offline approaches and online approaches. The former, offline log parsing approaches, require all logs are available before parsing and process them in a batch fashion. However, in real applications, logs are always collected in a streaming manner and applications have an increasing need for online monitoring and maintenance. [18]

Pipeline. An ordered sequence of processing stages (ingest → parse/tokenize → enrich → transform → buffer → export) that converts raw log events into structured, indexed records for querying, alerting, and downstream analytics. [19]

Heuristics. We can define them as practical rules, patterns or algorithms used to group, tokenize, or extract templates from log messages when exact formal methods are too slow or unavailable. [20]

Semantics (matching). This uses contextual attributes of the digital object to interpret the artifact in a manner that more closely corresponds with human perceptual categories. For example, perceptual hashes allow the matching of visually similar images and are unconcerned with the low-level details of how the images are persistently stored. Semantic methods tend to provide the most specific results but also tend to be the most computationally expensive ones. [21]

Machine Learning. Machine learning is the subset of artificial intelligence (AI) focused on algorithms that can “learn” the patterns of training data and, subsequently, make accurate inferences about new data. This pattern recognition ability enables machine learning models to make decisions or predictions without explicit, hard-coded instructions. [22]

LLMs. Large Language Models (LLMs) have evolved significantly, transitioning from early statistical and rule-based approaches to deep learning architectures powered by Generative Pretrained Transformers (GPTs). These models leverage self-attention mechanisms and large-scale datasets to generate human-like text, enabling advancements in natural language processing (NLP) tasks such as summarization, translation, content generation, and complex problem-solving. A key enhancement in LLMs is instruction tuning which specializes the pretrained LLMs to follow instructions provided in prompts. Instruction tuning enables effective in-context learning (ICL). Instead of modifying model parameters, ICL allows LLMs to learn from prompts that include instructions, examples, and queries, making them more adaptable to specialized applications [23]

6.2. Anchor study: a large-scale evaluation benchmark

This thesis uses the large-scale benchmark presented in “*A Large-Scale Evaluation for Log Parsing Techniques: How Far Are We?*” as the primary anchor for the related-work analysis. That study is particularly useful because it consolidates datasets, establishes common evaluation procedures, and compares a wide range of existing parsers in a single place. The benchmark improves on earlier standards by increasing dataset variety and scale, and by attempting to better reflect the heterogeneity and production characteristics of real-world logs [8]. For these reasons, the benchmark both guides the selection of test datasets and provides baseline results against which new or improved methods can be compared.

A more in-depth review of the benchmarks and datasets to be used in this thesis is provided in later sections, but for now, the benchmark’s main contributions can be summarized as follows:

- Datasets and scale: a curated collection of log datasets covering different domains and formats, chosen to better simulate real production conditions, such as high volume, template churn, and mixed structure.
- Evaluation protocol: common metrics and experimental settings that enable apples-to-apples comparisons between parsers.
- Comparative results: measured performance of multiple parsers across datasets, highlighting strengths and weaknesses of different approaches and revealing scenarios where existing techniques fail or degrade.

6.3. Taxonomy of parsing approaches and thesis plan

Following the benchmark’s taxonomy, this work adopts the same high-level classification of log-parsing techniques. The taxonomy partitions methods into four classes, as identified in the cited paper. For each class, this thesis will: (a) describe the core idea, and (b) select two representative parsers to analyze their algorithmic approach, strengths, and limitations, and to compare them with the other parsers.

6.4. Frequency-based Parsing

This type of method is founded on the intuition that tokens, which frequently occur within a specific log dataset, generally represent the static elements of those logs. Consequently, the extraction of frequent patterns provides a straightforward approach for automated log parsing. In detail, these log parsers first traverse the provided log dataset to construct frequent item-sets. Subsequently, these item-sets are utilized to derive the corresponding log template for log messages.

6.4.1. Logram: Efficient Log Parsing Using n-Gram Dictionaries.

Logram casts log parsing as a token-level, frequency-based problem using n-gram dictionaries. The method builds dictionaries of n-gram frequencies from the corpus and uses n-gram occurrence statistics to decide which token sequences are likely static template text (frequent n-grams) and which are dynamic variables (rare n-grams). Parsing then proceeds by scanning tokens and consulting the n-gram dictionary to assemble templates and extract parameters. The authors emphasize that dictionary construction is $O(n)$ and lookups are $O(1)$, making the method efficient; the algorithm also includes an automatic thresholding step to distinguish static vs dynamic parts. The design naturally supports incremental updates (online parsing) and parallel dictionary construction, which are used to scale to large logs. [24]

Logram is evaluated on the standard set of 16 public log datasets used in the LogPAI / Loghub benchmark (e.g., Android, Apache, BGL, Hadoop, HDFS, Linux, Mac, Spark, Windows, Zookeeper, etc.) [25]. The authors state that their tool and experimental data are shared on GitHub (repository: BlueLionLogram/Logram), so the implementation and experimental artifacts are publicly available. [26]

Now, when talking about strengths and limitations, Logram shows that it consistently outperforms prior parsers on runtime: the paper reports Logram is $1.8\times$ to $5.1\times$ faster than the next-fastest approaches in end-to-end parsing time across evaluated logs, with stable performance up to 1 GB experiments (dictionary build time included). This efficiency comes from cheap dictionary lookups and per-token processing rather than comparing entire lines. Another important factor that makes Logram a really solid parser is its online capabilities, the algorithm supports online updates (incrementally updating n-gram counts) and the authors show online mode achieves parsing results and efficiency close to offline mode, which is useful for real-time log ingestion.

And when talking about limitations, the authors acknowledge that evaluation covers 16 public datasets from the benchmark but cannot guarantee similar performance on arbitrary, unseen production logs. In particular, logs containing many unseen events or abrupt shifts (e.g., new event types appearing later in a stream) can reduce parsing quality unless the dictionary is updated.

And continuing with the problems that may appear later down the road when deploying this to a real production environment, is that the dictionary size grows with log volume and vocabulary; the authors report that lookups remain fast up to the experiments (≤ 1 GB logs), but very large or extremely diverse logs could increase memory demands and affect construction time (the paper checks up to 1 GB and reports stable behavior there). This is a practical consideration for very large-scale or high-cardinality logging systems.

The fast and resource aware approach that Logram takes into this problematic makes it a really solid contender to use it as the “statistical” part of the hybrid parser.

6.4.2. LogCluster - A data clustering and pattern mining algorithm for event logs.

LogCluster frames log parsing as a pattern-mining/clustering problem: it aims to discover frequently occurring line patterns (clusters) and identify outlier lines. The core observation is that every word that belongs to a pattern with support $\geq s$ must itself appear in at least s lines; LogCluster therefore begins by identifying frequent words (a user-specified support threshold s) and then builds cluster candidates by extracting, for each line, the ordered tuple of its frequent words. Candidates are maintained by counting supports and by adjusting per-candidate pattern metadata (wildcard ranges, variable min/max counts) as new lines are assigned; after candidate generation, candidates with support $< s$ are dropped and remaining candidates are reported as clusters (patterns). To scale frequent-word detection in very large logs, the method optionally uses a sketch of counters (hash-based counting) to filter out words that cannot be frequent, avoiding the need to store all unique tokens in memory. LogCluster also includes optional heuristics (e.g., Aggregate_Supports and cluster-joining heuristics) to reduce overfitting caused by low support thresholds and to merge overly specific candidates into more generic patterns. [27]

The paper evaluates LogCluster on several real-world logs (examples in the results table include “Authorization messages”, “UNIX daemon messages”, etc.) and reports both the number of clusters found and runtimes in direct comparison to the SLCT [28] baseline. The experimental section reports file sizes (lines), chosen support thresholds, cluster counts, and

runtime in seconds for LogCluster vs SLCT, illustrating behavior on medium-to-large log files. This comparison with only one log parser can hide multiple limitations, a more in depth benchmark and comparison with newer parsers would be required if it is decided to continue with this parser.

On the strengths of this parser we can mark that the sketch (hashed counters) used to pre-filter candidate words reduces the need to materialize all unique tokens for large logs, making the method viable on large files and reducing memory use in the frequent-word detection step.

And on the limitations hand, we can find that because LogCluster clusters by sets/tuples of frequent words and then creates patterns with wildcards, it is conceptually simpler than parsers that use syntactic position-based clustering or sophisticated token-level templates. As a result, on logs with many low-frequency tokens or highly variable message formats, pattern purity (precision/recall of templates) can degrade unless thresholds and heuristics are carefully tuned. The paper reports cluster counts and timings but does not present modern benchmark-style precision/recall numbers that are common in later log-parsing literature, so comparing "accuracy" requires caution.

And as mentioned before, a more robust benchmark with more types of logs, a higher quantity and a more hard comparison with state of the art parsers is necessary to review its performance, but with the current data it is not a strong candidate.

6.5. Similarity-based Parsing

These log parsers conceptualize log parsing as clustering logs into distinct clusters predicated on their similarity, and logs in each cluster share the same log template. Various methods employ different clustering algorithms (e.g., hierarchical clustering, density-based clustering) and definitions of similarity. Following the clustering process, log templates can be derived by extracting the common tokens from the logs within each respective cluster.

6.5.1. LogMine: Fast Pattern Recognition for Log Analytics.

LogMine [29] proposes an unsupervised, streaming framework for extracting log templates from large, heterogeneous log streams. The central idea is that real systems emit messages according to a relatively small set of templates; by clustering similar messages in one pass and extracting per-cluster templates, LogMine recovers those templates quickly while using little memory. The method begins with lightweight preprocessing (tokenization and normalization of common types such as timestamps, IPs and numbers), then applies a fast one-pass clustering scheme in which each incoming message is compared against cluster representatives using a simple field-wise distance. Comparisons employ early-abandoning heuristics to avoid unnecessary work, and clusters are maintained as compact representatives rather than storing all pairwise distances. For each dense cluster LogMine derives a template by merging token positions and distinguishing constant tokens from variable fields; the authors also produce a hierarchy of templates by iteratively relaxing the clustering threshold, yielding multiple abstraction levels (from specific to general) so operators can choose the granularity appropriate for monitoring or analysis.

The architecture includes a MapReduce-style parallelization so the algorithm can be deployed across workers for very large volumes.

In their evaluation the authors report experiments on a mix of public and proprietary datasets to demonstrate both scalability and template quality. The paper compares LogMine to exact but expensive baselines (e.g., density-based clustering followed by sequence alignment) and to prior unsupervised approaches, showing that LogMine preserves high-quality template extraction while drastically reducing runtime and memory requirements. The authors emphasize practical reproducibility concerns: the paper does not ship an official code release, so reproduction requires either re-implementation or use of community ports; however, the datasets used include standard public traces as well as large industrial logs that stress-test scalability.

LogMine’s principal strengths are its runtime and memory efficiency and its suitability for streaming contexts. Because it keeps only cluster representatives and uses one-pass assignment with early exits, its space footprint grows with the number of templates rather than with the square of the number of messages, making it practical for millions of log lines. The hierarchy of templates gives users control over specificity versus generality, a useful feature for different downstream tasks (e.g., anomaly detection versus summarization). Empirically, the authors report very large speedups over exact baselines on several workloads while maintaining competitive pattern quality.

The method has, however, notable limitations and trade-offs. Being a one-pass, order-dependent clustering method, its output can vary with the temporal ordering of messages; the authors argue that real-world logs tend to collocate similar messages in time (mitigating the issue), but the order sensitivity remains a theoretical weakness and can affect results on interleaved or adversarial streams. LogMine also requires tuning of a small set of clustering parameters (initial maximum distance and the relaxation factor used to build the hierarchy), and performance or template granularity can change with these settings. On datasets that lack repeatable structure or where messages are highly heterogeneous, template quality degrades and the algorithm requires careful parametrization. Finally, while a MapReduce variant enables parallelism, parallel speedups depend on workload characteristics and are not unlimited.

Compared to prior work, LogMine occupies a middle ground between slow but accurate alignment-based methods and lightweight rule- or tree-based parsers. In direct comparisons the paper shows that alignment-based baselines (which compute many pairwise distances) produce high-quality templates but at prohibitive memory and time cost, whereas LogMine achieves similar template quality at a fraction of the computational expense. Subsequent practical parsers (e.g., tree-based or heuristic methods) approach the same problem with different trade-offs—some favor deterministic, low-latency parsing while others use learned encodings, so LogMine’s contribution is its clear focus on scalable, algorithmic clustering combined with a practical multi-level template output that influenced later work in scalable log extraction.

6.5.2. *Length Matters: Clustering System Log Messages using Length of Words.*

Shima proposes LenMa [30], a lightweight, online method for automatically clustering system log messages and generating message templates by exploiting the lengths of tokens (words) within each log line. The core observation that motivates the method is empirical: messages produced by the same template often have the same sequence of token lengths even when token values vary (e.g., IDs, IP addresses, filenames), so the sequence of word lengths can act as a compact, discriminative signature for template identification. Based on this, LenMa encodes each log message as a vector/sequence of integer word lengths and performs one-pass clustering using these sequences as keys or signatures; in practice the algorithm groups messages with identical or similar length sequences and then applies simple template extraction (replacing variable token positions with wildcards) to produce human-readable templates. Because the method works on token lengths rather than token content, it avoids the costly two-pass, frequency-based preprocessing that some earlier techniques require and is naturally suitable for online/streaming log processing.

In his experimental section the author demonstrates the approach on several syslog collections (reported in the paper's tables) and shows that many templates are uniquely identified by their length-of-word sequences for typical system logs; the paper gives examples and statistics that justify the uniqueness hypothesis for common datasets. The evaluation compares LenMa qualitatively and quantitatively against prior template inference approaches (notably two-pass heuristics such as SLCT and other clustering methods), emphasizing LenMa's runtime simplicity and online capability. The paper is available on arXiv and has been cited in subsequent surveys and parser benchmarks that treat LenMa as an early online parser design.

Strengths. LenMa's main contributions are (1) simplicity: the representation (word-length sequences) is extremely cheap to compute and compare, (2) one-pass/online operation: the algorithm can cluster and output templates in a streaming setting without expensive preprocessing passes, and (3) practical effectiveness for many syslog types: for a range of server and network syslogs the length signatures are sufficiently discriminative to recover templates with low overhead. These properties make LenMa attractive for high-volume logging environments where low latency and low memory footprint are important.

Limitations and practical caveats. The approach trades content awareness for speed and compactness, which creates several limitations. First, collisions can occur: different templates may produce identical length sequences (for instance, when differing tokens happen to share lengths), causing over-merging of distinct templates unless extra checks are applied. Second, messages originating from the same template can sometimes vary in tokenization or include free-form text fields whose token lengths differ, causing under-clustering (splitting one logical template into multiple length-signature groups). Third, LenMa is inherently syntactic and language-/format-dependent: it assumes consistent tokenization and that important differences are captured by token length rather than by token content or positional semantics. Finally, while the paper reports promising results, it does not (in the arXiv version) ship a full open reference implementation for easy reproduction; follow-up toolkits and benchmarks (e.g., logparsing toolkits) include LenMa as a referenced method but often rely on community re-implementations for comparative evaluation. These limitations imply that LenMa is best used as a fast, first-

pass clustering technique or as a component in a hybrid pipeline (e.g., length-based grouping followed by content-aware refinement).

Relation to other work. LenMa sits among a family of log parsing methods that trade expressiveness for efficiency. Unlike two-pass frequency-based methods (SLCT) or heavy alignment/clustering approaches that compare token content pairwise, LenMa focuses on a single, compact signature that enables streaming execution. Compared to later production-oriented parsers (e.g., tree/heuristic parsers such as Drain or algorithmic clusterers like LogMine), LenMa emphasizes minimal state and speed; in many benchmark surveys it is cited as an early online parser design and as a useful baseline for speed-oriented scenarios, though practical parsing toolkits often combine length heuristics with token content features to get the best of both worlds

6.6. Heuristics-based Parsing

Another category of log parsers employs a diverse range of heuristic algorithms or data structures, such as the longest common subsequence-based approach, parsing trees, evolutionary algorithm, among others. These log parsers are designed to leverage the unique characteristics of log data to distinguish the templates and parameters in log messages.

6.6.1. Drain: An Online Log Parsing Approach with Fixed Depth Tree.

Drain [31] introduces a practical, streaming log parser that builds a fixed-depth parse tree to guide online template extraction and grouping. The core idea is to avoid expensive pairwise comparisons or deep unbalanced search structures by routing each incoming log message down a small-height tree whose internal nodes apply simple, rule-based splits (e.g., by token count, token position, or token type). At the leaves the algorithm maintains a short list of candidate clusters; an incoming message is compared only against those candidates (using token-level similarity rules) and either joins an existing cluster (with the cluster’s template updated if needed) or creates a new cluster. The tree is constructed and updated automatically from observed logs so that Drain adapts to new templates over time while keeping per-message processing time low.

The original paper evaluates Drain on multiple real-world datasets spanning web services, distributed systems, supercomputers, and standalone software to demonstrate both accuracy and efficiency at scale; the authors report high parsing accuracy across a diverse set of 11 datasets in follow-up evaluations. An important practical outcome is that Drain was implemented and released to the community (and later improved as Drain3), and it is included in popular log-parsing toolkits and benchmarks (e.g., LogPAI’s logparser toolkit and multiple open-source ports), which makes reproduction and direct comparison easy for follow-up research and practical deployment. Thus, both the paper’s experimental claims and accessible, maintained implementations exist for practitioners and researchers.

Drain offers several practical strengths that make it attractive for both research and production use. Designed for streaming operation, each log message is handled with a small, bounded amount of work, a short tree traversal followed by comparisons against a tiny candidate list, which yields very low latency and high throughput. In empirical evaluations and subsequent benchmarks Drain attains top-tier template extraction accuracy across many standard datasets, often surpassing earlier online parsers; its rule-based tree and simple token-level similarity checks make the algorithm deterministic and robust in practice.

Despite these advantages, Drain has trade-offs that users must consider. Its effectiveness depends on the heuristic splitting rules and token-similarity measures encoded in the tree; when tokenization is unusual, free-text fields are prevalent, or message formats are highly irregular, those heuristics can misclassify or fragment templates unless adjusted or augmented. Leaves of the fixed-depth tree can also accumulate many candidate templates for systems with very large or heterogeneous template sets, increasing per-message comparison cost and partially eroding the method’s bounded-work promise. More generally, Drain assumes a degree of stability in token positions and counts so that tree-based splits are meaningful; logs that violate these structural assumptions (for example, with many optional tokens or highly variable natural-language content) challenge the approach and typically require per-normalization, additional filtering, or hybrid pipelines to restore high parsing fidelity.

Viewed in the broader landscape of log parsing techniques, Drain occupies a practical middle ground between extremely lightweight heuristics and heavyweight alignment or clustering methods. It deliberately trades some theoretical generality for a large gain in throughput and operational simplicity: compared with alignment-based or pairwise-distance approaches, Drain discards costly global comparisons and therefore scales far better in streaming contexts, while compared with minimal heuristics (e.g., word-length signatures) it usually attains substantially higher template fidelity due to its structured candidate search and token-aware checks. This combination of speed, accuracy, and reproducible implementations has led to wide adoption in both research benchmarks and industry toolchains; in practice Drain is often used as a reliable baseline or as a core component within hybrid systems where light preprocessing or downstream refinement compensates for its heuristic assumptions.

6.6.2. Abstracting Execution Logs to Execution Events for Enterprise Applications (Short Paper).

M. Jiang and their team [1] propose a practical, largely automated approach to abstract execution logs into execution events so that large enterprise applications can be monitored and analyzed without source-code instrumentation. The paper begins from the observation that raw log lines are poor monitoring artifacts because each occurrence of an event contains dynamic fields (IDs, amounts, timestamps, lists, etc.), causing every instantiation to look different even though they originate from the same underlying execution event. To address this, the authors present a lightweight pipeline that first anonymizes and tokenizes log lines, distinguishes static (constant) and dynamic parts using a small set of general heuristics, and then groups lines that share identical static structure into execution-event bins. Where needed, the method applies clone-detection style matching to reconcile slightly differing static fragments that nonetheless represent the same event. The authors also describe reconciliation and bin-merging steps to handle multi-statement events (where a single execution event emits several log

statements) and optional post-processing to build an execution-event database usable for monitoring and summary queries. This approach deliberately minimizes the need for heavy system knowledge and avoids intrusive instrumentation, instead leveraging the abundance of existing logs to infer operational events.

The paper validates the approach with a substantive case study on logs from multiple large enterprise applications (the short paper and its extended journal version report experiments on logfiles from four enterprise systems developed across three organizations, including two datasets contributed by Research In Motion). Evaluation uses standard information-retrieval metrics (precision and recall) computed by comparing the inferred event bins against ground-truth abstractions; the authors report that their method attains high precision and recall across the case-study datasets, demonstrating effective abstraction even when logs do not follow a strict format. The paper also compares the approach to SLCT, a commonly used pattern-finding tool, and shows that the authors' heuristics and reconciliation steps produce more faithful event abstractions for the enterprise traces studied. The original short paper does not publish an official, publicly hosted implementation, but the authors later expanded the method in a journal article that provides fuller algorithmic detail and results, and the work has been widely cited and referenced in subsequent log-abstraction and parsing literature.

The principal strengths of the approach are its practicality and low intrusiveness: by working from already-available execution logs, it avoids the operational costs and deployment friction of instrumentation and profiling; by using simple, general heuristics the method requires much less system expertise than fully rule-based systems. The paper demonstrates that these heuristics are sufficient to recover event structure with high fidelity in realistic enterprise settings, and that the technique scales to large log volumes because it groups lines by static structure rather than performing expensive pairwise comparisons. Another strength is that the approach explicitly addresses multi-statement events and provides reconciliation heuristics, a frequent real-world complication that many simpler parsers ignore.

There are important limitations and trade-offs to note. First, the approach depends on heuristic detection of static versus dynamic tokens; when logs contain unstable tokenization, extensive free-form natural language, or inconsistent coding practices across versions, the heuristics can either over-split (fragment one logical event into several bins) or over-merge (collapse distinct events that coincidentally share static substrings). Second, while the method reduces the need for system expertise, some manual tuning or human validation is still needed to reconcile ambiguous bins and to parameterize reconciliation thresholds for different applications. Third, the short paper itself does not provide a public tool release, which means practitioners must re-implement the heuristics or rely on the extended journal version and follow-up work for replication. Finally, because the grouping is primarily based on static structure, the approach is best at producing event types rather than extracting fine-grained semantic meaning from dynamic fields (e.g., the exact reason code embedded in a free-text message), so downstream tasks that require rich semantic extraction may need additional analysis layers.

6.7. Semantic-based Parsing

In recent years, numerous parsers have employed deep neural networks to understand the semantic meaning of logs, thereby improving parsing accuracy. In detail, these log parsers employ supervised methodologies, utilizing models such as bidirectional long short-term memory or pre-trained language models to learn the semantic information of log messages, thereby distinguishing between log templates and parameters through the completion of classification tasks.

6.7.1. *SemParser: A Semantic Parser for Log Analytics*

SemParser [32] is a semantic-aware approach to log parsing that goes beyond syntax-only template extraction by explicitly mining and leveraging the semantics embedded in log messages. The method is organized in two stages: an end-to-end semantic miner that identifies explicit semantic entities inside individual log messages (e.g., resource IDs, operation types, status indicators), and a joint parser that uses a contextual knowledge base to infer implicit semantics and produce structural outputs (templates plus labeled parameters). The joint parsing stage combines the local semantic cues with global contextual knowledge so that structural parsing and semantic inference inform each other, producing templates annotated with meaningful semantic roles rather than raw placeholders. This design lets SemParser output richer, machine-actionable structured logs that carry both template structure and semantic labels suitable for downstream analytics.

The authors evaluate SemParser on a variety of real-world systems and downstream tasks. For semantic-mining quality the paper reports very high extraction accuracy on semantics derived from log messages collected across six widely used systems; for downstream impact the authors show that injecting SemParser-extracted semantics improves anomaly-detection models by between 1.2% and 11.7% on two anomaly datasets, and improves failure-identification performance by about 8.65% on a dedicated dataset. The work was published at ICSE 2023 and the authors provide a replication package (code and data) on GitHub, enabling reproducibility and direct application of their models in follow-up studies [33].

SemParser’s principal strength lies in its semantic orientation: by treating log parsing as a combined semantic-labeling and structural extraction problem, it recovers richer information than syntax-only parsers, which in turn measurably benefits downstream tasks that rely on meaningful features rather than opaque parameters. The two-stage architecture (local semantic miner + joint parser) allows SemParser to capture both explicit tokens (clearly typed parameters) and contextual, implicit semantics (roles inferred from surrounding logs or a learned knowledge base), improving both the interpretability and utility of parsed outputs. The availability of an author-maintained replication package further strengthens its practical value for researchers who need a semantically informed baseline or who want to adapt the technique to new systems.

There are, however, important limitations and trade-offs to consider. SemParser relies on supervised or semi-supervised learning components to identify semantic entities and to build the contextual knowledge base, which introduces labeling cost and potential sensitivity to the domains represented in training data; generalizing to novel log formats or domains not seen in training may require additional annotation or domain adaptation. The model’s complexity and reliance on learned components make SemParser heavier than light-weight, rule-based online parsers (e.g., Drain) and may entail higher

runtime or resource costs during training and, depending on implementation choices, at inference time. Finally, while the reported downstream improvements are convincing on the evaluated datasets, the benefits will vary with the downstream model and task, in scenarios where downstream pipelines already engineer strong features from raw parameters, the marginal gains from semantic labels may be smaller.

6.7.2. Log Parsing with Prompt-based Few-shot Learning

Log Parsing with Prompt-based Few-shot Learning or *LogPPT* [34] is a prompt-based, few-shot log parsing framework that adapts pre-trained language models to the task of template extraction and parameter identification using only a small set of labeled examples. The approach reframes parsing as a parameter recognition problem: a prompt-tuning procedure (specifically a template-free virtual-label prompt technique) is used to teach a frozen pre-trained model to predict whether each token in a log message is a keyword (constant) or a parameter (variable). To make few-shot learning practical, LogPPT includes an Adaptive Random Sampling procedure that selects a compact but diverse set of K labeled messages for annotation; during training the method generates a virtual label token and applies a customized masked-language-model objective so that the model learns to mark parameter spans in a single pass. The pipeline also contains lightweight preprocessing and an online parsing module with a caching mechanism to support efficient inference in streaming settings.

The authors evaluate LogPPT on a broad benchmark of 16 public log datasets, measuring parsing accuracy and runtime against conventional parsers and several prompt-tuning variants. Results reported in the paper show that LogPPT can achieve strong parsing accuracy and that its template-free prompt tuning is both more sample-efficient and faster than enumerative prompt schemes or naive fine-tuning in many settings. The project is reproducible: the authors published a companion repository that implements LogPPT, including the sampling routine, prompt tuning, and inference/caching code, which simplifies replication and practical adoption.

LogPPT’s principal strengths are its semantic transfer from large pre-trained language models (PLMs) and its low labeling cost. By relying on PLMs, the method exploits contextual and semantic knowledge that heuristic pattern matchers miss, which helps it identify parameters and distinguish keywords across heterogeneous formats without dataset-specific regular expressions. The adaptive sampling strategy reduces manual annotation burden while improving representativeness of the few labeled shots, and the template-free prompt design avoids costly span enumeration at inference time, yielding a practical few-shot trade-off between accuracy and annotation effort. The availability of an open implementation further increases the method’s value as a reproducible, research-grade baseline for ML-driven log parsing.

Important limitations and caveats temper these advantages. Although LogPPT is few-shot, it still requires dataset-specific labeled examples and the adaptive sampler’s quality affects downstream performance; in new domains one must still label K examples and verify sampler coverage. The reliance on pre-trained language models brings compute and latency costs (especially for large models) and can make online, high-throughput deployment more expensive than lightweight heuristic parsers unless smaller or optimized inference stacks are used. Prompt-tuning techniques can also be sensitive to noise and to tokenization peculiarities of log data; the authors note threats to validity tied to dataset quality and show that, with more

labeled data, standard fine-tuning can match LogPPT, implying that the few-shot advantage may shrink as labeled data grows. Finally, like other ML approaches, LogPPT’s generalization is constrained by its training examples and the pre-training domain of the backbone model, so extreme or novel log formats may still require additional adaptation.

6.7.3. UniParser: A Unified Log Parser for Heterogeneous Log Data

UniParser [35] presents a deep-learning approach to log parsing that explicitly targets heterogeneous log sources by learning common logging “syntax” across multiple systems and then applying the learned model to new sources. The key architectural idea is to cast parsing as a token-level classification problem (template token vs parameter token) and to train a model that fuses token-local features with context encodings and contrastive context similarity to emphasize shared logging patterns. Concretely, UniParser contains three main modules: a Token Encoder (character-level / token embedding) that captures token-intrinsic cues, a Context Encoder (Bi-LSTM over neighboring token embeddings) that captures ordering and local context, and a Context Similarity module used during offline training to force context embeddings of similar-pattern messages closer in representation space (contrastive loss). Training is done cross-source: the model is trained on labeled logs from many different systems so it can learn logging conventions (e.g., parameters often follow “=” or appear inside parentheses) that generalize beyond any single application. At inference time the trained model labels tokens in new logs in parallel and reconstructs templates by grouping tokens predicted as template tokens versus parameters, enabling an online parsing flow after an offline cross-source training phase.

The authors evaluate UniParser on a comprehensive benchmark drawn from public repositories (the LogPai / LogHub collections): 16 public datasets spanning distributed systems, supercomputers, operating systems, mobile systems, server applications and standalone software. Experiments use token-level transformed labels (the paper notes some label calibration during preprocessing) and report both Group Accuracy (a standard clustering-style metric) and a Message-Level Accuracy metric proposed by the authors to better reflect correct template+parameter extraction. UniParser achieves large gains over prior parsers in the paper’s evaluation, roughly +12% on Group Accuracy and $\approx$ +40% on Message-Level Accuracy compared with state-of-the-art baselines, and the authors report very fast inference (parsing millions of logs in 2 to 3 minutes, about half the time of the most efficient existing parser in their comparison). The implementation used PyTorch and GPU training/inference; in practice community benchmark toolkits (e.g., the LogHub-2.0 benchmark) include UniParser implementations and one-click evaluation scripts, which researchers commonly reuse for reproduction.

UniParser’s principal strengths are its cross-source generalization and its ability to learn semantically meaningful logging conventions rather than relying on brittle surface heuristics. By training across many sources and using context-level contrastive learning, the model captures recurring structural cues (bracketing, separators, status-position patterns) that recur across projects; this allows direct application to new log sources with little or no re-labeling in many cases and yields much higher message-level parsing fidelity than many heuristic or clustering parsers. The architecture’s separation of token-level and context-level encoders also provides robustness to content diversity while letting the Context Similarity module compress diverse but semantically similar contexts into a compact representation space. Finally, the authors demonstrate

real-world applicability: UniParser has been applied inside large industrial environments (Microsoft 365 and Azure scenarios are mentioned), indicating practical utility beyond benchmarks.

There are trade-offs and limitations to bear in mind. UniParser is a trained, supervised model: it requires labeled log data for the offline training phase (the paper uses labeled datasets from LogHub), so there is upfront labeling cost and the need for GPU resources during training and, depending on deployment constraints, for efficient inference (the experiments used an NVIDIA P100 GPU). While the authors show good cross-source transfer, some distinctive log sources still require a small amount of fine-tuning. The model also depends on the tokenization choices made before training (splitting on spaces, commas, etc.), and rare tokenization edge cases (e.g., parentheses that should stay attached to tokens) can produce misclassifications unless additional preprocessing is introduced. From a systems perspective, UniParser trades runtime/model complexity for accuracy: it is heavier than light-weight heuristic parsers (Drain, LenMa) and therefore may be less attractive when extremely low-latency, CPU-only parsing is required.

6.8. General review of LLM approaches

To conclude the semantic parsers review, and to have a more wide view into this emerging world of using LLMs to contribute to the Log Parsing problem, it is important to review a paper that compares 29 LLM-based log-parsing approaches and benchmark a representative subset, synthesizing the common design choices, evaluation pitfalls, and practical trade-offs that characterize this literature.

The work that we can find in “System Log Parsing with Large Language Models: A Review” highlights that LLM-based parsers are not merely a novelty but a practical alternative that frequently reduces manual configuration while offering strong accuracy and generalization across heterogeneous log formats [7].

The principal strengths of LLM-based (semantic) parsers are (1) semantic generalization: LLMs can learn to separate template (static) parts from variable fields by leveraging contextual and distributional language knowledge rather than brittle syntactic heuristics; (2) reduced need for hand-crafted parameters: many LLM solutions require fewer tuning knobs compared to classic parsers; and (3) flexible learning paradigms: both in-context learning (ICL) and selective fine-tuning have been shown effective, and can be combined with methods such as retrieval-augmented generation (RAG), caching, and template-revision to improve throughput and stability.

A practical middle ground emerging from the literature is hybridization: combine LLM strengths with lightweight, high-throughput components (for example, use fast rule/statistics-based filters or batching layers before invoking an LLM, or employ RAG and cached templates to minimize calls). Empirical studies demonstrate that such designs can approach the runtime of classical parsers while retaining the semantic gains of LLMs; nonetheless, careful engineering and domain-aware prompt/architecture choices are required to balance cost, latency, and accuracy.

6.9. Evaluation Criteria and Datasets Commonly Used

A robust and reproducible evaluation protocol is essential for comparing log parsers because parsing performance depends strongly on dataset scale, template diversity, event rarity, and preprocessing choices. Historically the community relied on the original Loghub collection (a curated set of real-world logs from many domains) [36], but concerns about scale, representativeness and inconsistent template labels motivated a newer, more rigorous benchmark: LogHub-2.0 [25].

LogHub-2.0 is an expanded, manually annotated, large-scale dataset collection designed specifically for fairer, more practical evaluation of log parsers. The collection contains 14 datasets drawn from different software domains with millions of log lines per dataset (on average ~3.6M lines) and explicit template/parameter annotations. The authors also re-evaluated a broad set of state-of-the-art parsers on this collection and introduced an evaluation protocol (including a new metric) intended to reduce bias caused by highly imbalanced template distributions and to surface parser behavior on rare system events, a crucial practical concern for diagnostics. LogHub-2.0 therefore shifts the evaluation focus from small, heterogeneous testbeds toward larger, production-style workloads that better reflect deployment challenges.

Not every paper uses LogHub-2.0; alternatives and complementary resources include the original Loghub collection [36] (the earlier curated set of diverse real-world logs) and the various named datasets it contains (HDFS, BGL, Apache, Android, OpenStack, etc.). These individual datasets remain popular because they are well-known and easy to reproduce experiments on. Another alternative that some papers used are private/industrial logs [1]; these datasets are often not publicly available for confidentiality reasons, which complicates reproducibility.

Because of the diversity of these practices, the literature frequently suffers from incomparable results: different preprocessing, different template-matching tolerances, and divergent success criteria (exact template match vs. token-level correctness) make cross-paper comparisons difficult. LogHub-2.0 explicitly aims to reduce this problem by offering large annotated sets and a clearer evaluation protocol.

This evaluation protocol is based on 4 basic metrics that balances message-level correctness, template-level fairness, and practical efficiency so that high reported accuracy cannot be achieved by merely solving a few very frequent templates. These four metrics can be grouped into two categories, “Message-level metrics” and “Template-level metrics”, here is a more in depth description of each one.

Message-level metrics (line-weighted correctness). These quantify how many log messages are correctly parsed or grouped. During this thesis, the following 2 are the ones that have the highest weight. First Grouping Accuracy (GA) which is the proportion of log messages that the parser assigns to the correct group (for example, the parsed group for a message equals the ground-truth group). Then Parsing Accuracy (PA) which is the proportion of log messages whose template tokens and parameter tokens are all correctly identified (every token, static and variable must match the ground truth). These metrics emphasize correctness on high-volume message classes and are useful when the goal is to maximize overall line-level correctness.

Template-level metrics (per-template fairness) are present in this thesis because log templates are typically long-tailed, and so message-level averages can hide poor performance on many rare templates. To mitigate this, LogHub-2.0 introduces template-level evaluation that treats each unique template equally.

First F1 of Group Accuracy (FGA) which is a template-level F1 that evaluates grouping correctness at the template granularity. A parsed template that exactly matches a ground-truth template's message set counts as a true positive. Precision and recall are computed over templates (parser-produced vs. ground truth) and combined into an F1 score (FGA). FGA therefore prevents parsers from being over-rewarded for doing well only on highly frequent templates. Specifically, FGA is the harmonic mean of PGA (Precision of Group Accuracy) and RGA (Recall of Group Accuracy). Let N_p be the number of templates that are generated by a log parser, and N_c be the number of templates that are correctly parsed by the log parser. The correctness here has the same definition as in GA, i.e., a log template is considered as correctly parsed if and only if the set of log messages belonging to this template matches the set indicated in the ground truth. N_g is the actual correct number of templates in the ground truth. Based on these notations, we can define

$$PGA = \frac{N_c}{N_p}$$

Figure i: Equation for the Precision of Group Accuracy

and

$$RGA = \frac{N_c}{N_g}$$

Figure ii: Equation for Recall of Group Accuracy

Then, we can calculate FGA as their harmonic mean,

$$FGA = \frac{2(PGA)(RGA)}{PGA + RGA}$$

Figure iii: Equation for the F1 of Group Accuracy

Then F1 of Template Accuracy (FTA) is the template-level F1 for exact template reconstruction (a stricter template correctness metric). For FTA a parsed template counts as correct only if it corresponds to a single ground-truth template and the parsed template tokens match the ground truth exactly, we can call that number of correctness as $\hat{N}_c$. FTA measures how well a parser reconstructs the canonical template text. Then, we can define PTA as

$$PTA = \frac{\hat{N}_c}{N_p}$$

Figure iv: Equation for Precision of Template Accuracy

and RTA as

$$RTA = \frac{\hat{N}_c}{N_g}$$

Figure v: Equation for Recall of Template Accuracy

And FTA can be calculated as their harmonic mean,

$$FTA = \frac{2(PTA)(RTA)}{PTA + RTA}$$

Figure vi: Equation for F1 of Template Accuracy

Apart from those 4 well-defined metrics, the benchmark explicitly records runtime behavior, completion failures, and resource requirements so accuracy alone cannot mask impractical parsers. The LogHub-2.0 study documents that many parsers fail to finish all datasets within a reasonable timeframe (the paper reports multiple parsers not completing within the experiment's limit), and semantic/learning-based methods often require GPU resources that affect deployability in CPU-only environments. The authors also note that running the full large-scale benchmark is time-consuming and provide scripts and results to reproduce experiments. These operational measurements (runtime, completion, hardware needs) are reported alongside accuracy so readers assess deployability, not only theoretical performance.

For this reason, is that this thesis will use LogHub-2.0 to benchmark any parser that will be built and/or tested. This way we can compare with metrics already stated and we can get a clear view in how the work done is different from the current state of the art.

6.10. Open challenges on the current state of the art

Try to combine semantic and statistical information. Semantic-based log parsers possess superior capabilities in distinguishing parameters from templates, which also substantiates the significance of semantic information in the process of log parsing. However, their grouping abilities are compromised due to the neglect of global information. This is inevitable, given that these log parsers exclusively process each log message in isolation. A potential avenue for future research could involve the combination of semantic and statistical information in logs, thereby simultaneously enhancing the parsing and grouping capabilities. Extracted from “A Large-Scale Evaluation for Log Parsing Techniques: How Far Are We?” [8]

7. Investigation Method

7.1. Research Design

This thesis will follow a quantitative, quasi-experimental research design. The choice of a quasi-experimental approach is justified because I will actively intervene in and configure log-parsing systems, but full random assignment of real-world logs or environments is not possible. The study therefore manipulates independent variables (for example: parser configuration choices, preprocessing steps, or hybridization strategy) and measures the effects on well-defined quantitative outcomes with the metrics explained in earlier sections.

Experiments will be run using a secondary data source, the LogHub benchmark. This benchmark provides a diverse collection of labeled log datasets and standard evaluation tasks. Using this benchmark ensures that results are reproducible and comparable to prior work. Performance will be quantified with established metrics, and statistical summaries will be reported so differences can be evaluated accordingly.

The study does not involve human subjects, surveys, or qualitative data collection. All observations derive from the deterministic output of software systems operating on fixed input datasets, which ensures full reproducibility of the experimental conditions. No repeated trials are conducted per configuration; each parser is run once per dataset, and results are taken as-is.

7.2. Framework Pipeline Overview

The overall experimental pipeline followed in this thesis is structured as four sequential stages: parser selection and modification, testing and benchmarking with LogHub, comparison with the state of the art, and analysis leading to final conclusions.

In the first stage, candidate parsers are evaluated against established benchmark results to identify suitable components for the hybrid system. This selection process, documented in Section 8.1, considers both parsing accuracy metrics and computational characteristics. The selected parsers are then modified or integrated as needed to construct the DrainUP hybrid pipeline, whose architecture and implementation are described in detail in Chapter 8.

In the second stage, the constructed system (along with its unmodified baselines) is evaluated against the LogHub 2.0 benchmark [25]. Two benchmark configurations are used: the 2k configuration, which evaluates each parser on a fixed sample of 2,000 log lines per dataset, and the full configuration, which uses the complete annotated log lines. Both configurations are evaluated across 13 log datasets drawn from diverse real-world systems, covering application logs, distributed system logs, and operating system logs. The specific datasets and benchmark configurations are described in Chapter 9.

In the third stage, the results obtained for DrainUP are compared against its two direct baselines (the unmodified Drain parser [31] and the unmodified UniParser [35]). This comparison allows the hybrid system's performance to be situated within the current landscape of log parsing research.

Finally, in the fourth stage, the results are analyzed with respect to the research questions stated in Chapter 5. This includes examining the accuracy trade-offs introduced by the confidence routing mechanism, the distribution of parsing workload between the two components, and the degree to which the hybrid approach generalizes across different log formats without retraining or reconfiguration.

8. Methodology and Implementation

8.1. Parser Selection for the Hybrid Approach

Selecting the right pair of parsers is a critical step toward a hybrid log-parsing solution that leverages the complementary strengths of statistical and semantic approaches. This section frames parser selection as a multi-criteria decision problem: statistical (similarity/frequency/heuristic) methods excel at high-throughput grouping of common templates, while semantic (ML/knowledge-driven or LLM-based) methods can disambiguate rare, context-dependent events that statistical rules miss.

To choose candidates for the hybrid solution, I will evaluate parsers based on the metrics presented in the previous section 6.9., but not just those metrics. A complementary input will be used to also decide which parser to select, this additional variable is the time elapsed to complete the parsing of the whole LogHub 2.0 benchmark (14 datasets in total). In their report, this is also a critical variable to evaluate the performance of the parsers.

Table i: Table with the most important metrics used within the LogHub benchmark

Parser	GA	PA	FGA	FTA
Logram	0.340	0.200	0.200	0.050
LogCluster	0.570	0.110	0.040	0.010
LogMine	0.750	0.240	0.370	0.120
LenMa	0.800	0.240	0.460	0.160
Drain	0.840	0.470	0.550	0.280
UniParser	0.660	0.680	0.500	0.260
LogPPT	0.560	0.760	0.590	0.490

Table ii: Efficiency metrics, such as time spent in average per dataset, a hint to its slowest dataset and if the given parser actually completed parsing all the datasets (given a 12 hours limit)

Parser	Time spent in Average per Dataset	Slowest Dataset	Completed all Datasets?
Logram	946.950s	5290.140s (OpenSSH)	No
LogCluster	200.660s	1082.260s (Thunderbird)	Yes
LogMine	4254.820s	17677.970s (Hadoop)	No
LenMa	1370.540s	6901.860s (HPC)	No
Drain	425.360s	2491.790s (Thunderbird)	Yes
UniParser	2225.250s	15368.390s (Thunderbird)	Yes
LogPPT	4109.840s	27008.590s (Thunderbird)	Yes

Table ii summarizes efficiency metrics extracted from the LogHub-2.0 benchmark [25] and the benchmark evaluation paper [8]. All times were reported in the original CSV in seconds. The benchmark used a 12 hours limit for a given parser to finish processing all 14 datasets, in the instance that a parser was not able to complete all of them I added the fourth column to table ii marking if the given parser completed all 14 datasets. The specific and original CSV tables with the effectiveness and efficiency metrics can be found in their GitHub repository [37]

Based on the benchmark results presented above, I select Drain as the statistical component and UniParser as the semantic component for the hybrid framework. Drain delivers the best efficiency among parsers that completed the full LogHub-2.0 benchmark, with an average parsing time of 425.36 s (≈ 7.09 min) per dataset and a maximum observed dataset time of 2491.79 s (≈ 41.53 min). UniParser provides a substantially stronger parsing accuracy profile on the same benchmark (notably higher PA than Drain), while still completing all datasets; its average parsing time is 2225.25 s (≈ 37.09 min) and its slowest dataset run required 15368.39 s (≈ 256.14 min, ≈ 4.27 h). For comparison, LogPPT shows higher average runtimes (4109.84 s ≈ 68.50 min) and a much larger slow-case time (27008.59 s ≈ 450.14 min ≈ 7.50 h), which makes UniParser a better fit for environments with limited compute power.

The selection shown combines effectiveness and operational constraints. Drain’s tree-based, online grouping is fast and robust for high-volume, frequent templates, matching the role of a first-stage parser in a cascade hybrid; UniParser’s higher template/parameter accuracy makes it well-suited to resolve ambiguous or low-frequency cases that Drain might not be able to handle. This choice balances accuracy gains on rare/semantic events with practical compute limits: Drain minimizes overall cost by handling the majority of lines quickly, while UniParser is invoked selectively to refine difficult cases. The hybrid is therefore designed to maximize coverage and downstream utility while keeping average processing time and resource needs within realistic bounds for the target environment.

8.2. System Architecture Overview

DrainUP is a hybrid log parsing system implemented in Python that combines a statistical parser, Drain [31], with a semantic parser, UniParser [35], through a confidence-based routing mechanism. The core design principle is that the statistical component handles the majority of log lines efficiently, while the semantic component is invoked selectively for lines that the statistical parser could not match with sufficient confidence. This division of labor aims to preserve the speed advantage of statistical parsing while leveraging the generalization capability of semantic parsing where it is most needed.

The system operates as a two-stage pipeline, illustrated in Figure vii. In the first stage, a modified version of Drain processes the entire input log dataset. As each line is parsed, Drain's internal matching function produces not only a template assignment but also a numerical confidence score reflecting the quality of that match. Lines that receive a confidence score below a configurable threshold are flagged and written to a separate file, isolating the subset of log lines that Drain could not handle reliably. In the second stage, UniParser is executed exclusively on this low-confidence subset, applying its semantic model to assign templates to the lines that Drain struggled with. The outputs of both stages are then merged into a single structured result, which is passed to the evaluation pipeline.

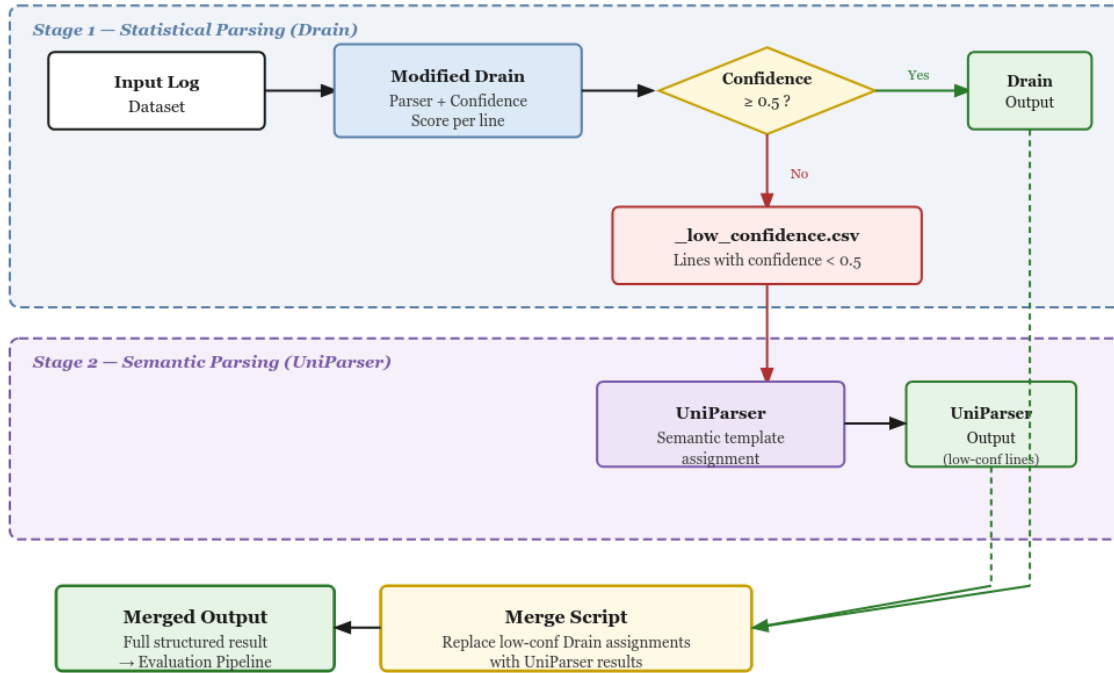


Figure vii: High-level architecture Diagram of the pipeline used by DrainUP

It is important to note that this two-stage, file-based architecture was adopted specifically to maintain compatibility with the LogHub 2.0 benchmark evaluation framework [37]. In a production or real-time deployment scenario, the same logical pipeline could be implemented as a real time parsing system, where each incoming log line is routed to the appropriate parser on the fly without intermediate file. The file-based design is therefore an artifact of the evaluation environment rather than a fundamental architectural constraint.

8.3. Modified Drain: Confidence-Aware Parsing

The first stage of the DrainUP pipeline is built on a modified version of Drain [31], a widely used statistical log parser that organizes log templates in a fixed-depth prefix parse tree. In its original form, Drain processes each incoming log line by traversing the tree based on log length and leading tokens, retrieves a list of candidate template clusters, and selects the best match using a token-by-token similarity function. If the similarity of the best candidate meets or exceeds a similarity threshold then the line is assigned to that cluster; otherwise, a new cluster is created. The original implementation returns only the matched template, discarding the intermediate similarity score after the matching decision is made.

The key modification introduced in DrainUP is the preservation and propagation of this similarity score as an explicit confidence value. This change is located specifically in two methods: *seqDist()* and *fastMatch()* [38].

The *seqDist()* method computes the similarity between a template sequence and an incoming log line token by token. Wildcard tokens (<*>) in the template are skipped and counted separately as parameter positions. For all remaining positions, an exact string match is required to increment the similarity counter. The similarity score is then computed as the ratio of matching tokens to the total length of the template, as shown in the following Equation:

$$confidence = \frac{simTokens}{|template|}$$

Figure viii: Equation for Similarity Score equation used in Drain's code

A score of 1.0 therefore indicates that every non-wildcard token in the template matched the incoming line exactly, while a score of 0.0 indicates either that no cluster was found.

The *fastMatch()* method iterates over all candidate clusters in the retrieved list and selects the one with the highest similarity score, using the number of wildcard positions as a tiebreaker. Crucially, the modified implementation returns both the matched cluster and its associated similarity score, rather than the cluster alone. This score is then stored alongside the parsed result in the output DataFrame as a Confidence column, making it available for downstream routing decisions.

Beyond these two methods, three further modifications were made to the original Drain codebase. First, the main *parse()* method was extended to maintain a *confidence_scores* list in parallel with the main parsing loop, recording the confidence value produced by *fastMatch()* for every log line processed. Second, a new method, *output_low_confidence_logs()*, was introduced to filter the parsed output by this confidence list. Any line whose confidence score falls below the configurable *confidence_threshold* parameter (default 0.5) is written to two separate files: a UniParser-compatible CSV containing the original log columns, and an auxiliary file retaining the Drain-assigned *EventId* to facilitate the subsequent merge step. Third, the output DataFrame is augmented with the Confidence column, providing full traceability of the routing decision for every line in the dataset.

8.4. Low-Confidence Routing and UniParser Integration

Once the modified Drain parser has completed its pass over the full input dataset, the routing mechanism is applied to separate log lines into two disjoint subsets: those for which Drain produced a high-confidence template assignment, and those for which it did not. This separation is performed by the `output_low_confidence_logs()` method, any line whose confidence score falls strictly below this threshold is written to the low-confidence output file; all remaining lines are retained in Drain's primary structured output.

The low-confidence file is written in a format compatible with UniParser's expected input structure, preserving the original log columns without modification. A second auxiliary file is written in parallel, containing the same low-confidence lines alongside their Drain-assigned EventId values. This auxiliary file plays no role in UniParser's processing but is essential to the merge step described in Section 8.5, as it provides the index mapping required to reintegrate UniParser's results into the correct positions of the final output.

UniParser [35] is then executed on the low-confidence file as a self-contained process. UniParser is a semantic log parser that employs a pre-trained language model to identify recurring patterns in log messages, grouping lines into templates based on semantic similarity rather than token-level string matching. Because UniParser operates on meaning rather than exact token overlap, it is better equipped to handle log lines that contain highly variable content, rare token sequences, or structural patterns that differ significantly from those already observed in Drain's template tree. These are precisely the characteristics that tend to produce low confidence scores in Drain's matching function, making UniParser a natural complement for this subset.

The output of UniParser is a structured file containing template assignments for each of the low-confidence lines. This output, combined with Drain's high-confidence structured output, forms the two inputs to the merge pipeline, which is described in the following section.

8.5. Merge Pipeline

The merge pipeline is responsible for combining the outputs of the two parsing stages into a single, coherent structured result that covers the full input dataset. It is implemented as a separate Python script that is executed after both Drain and UniParser have completed their respective passes. The merge script takes three inputs: Drain's primary structured output, containing high-confidence template assignments for the majority of log lines; UniParser's structured output, containing template assignments for the low-confidence subset; and the auxiliary file produced during the routing step, which maps each low-confidence line back to its original position in the dataset via the Drain-assigned *EventId*.

The merge procedure operates as follows. The script first loads Drain's full structured output as the base DataFrame, which at this point contains template assignments for all lines in the input dataset, including those that were flagged as low-confidence. The merge step replaces these provisional assignments with UniParser's results by joining the two outputs on the line index preserved in the auxiliary file. For each low-confidence line, the Drain-assigned EventId and template are overwritten with the corresponding values from UniParser's output. Lines for which Drain produced a high-confidence assignment are left unchanged.

The result of this procedure is a single DataFrame covering all lines in the input dataset, where high-confidence lines carry Drain's template assignments and low-confidence lines carry UniParser's template assignments. This merged DataFrame is then written to disk in the format expected by the LogHub 2.0 evaluation framework [37], which compares the parser's assigned templates against the ground-truth annotations to compute the evaluation metrics described in Chapter 6.9.

One practical consideration in the merge step is index alignment. Because UniParser processes only a subset of the original log lines, the line indices in its output do not correspond to a contiguous range and cannot be assumed to match the row order of Drain's output directly. The auxiliary file written during the routing step preserves the original positional indices of the low-confidence lines within the full dataset, and the merge script uses these indices explicitly to ensure that each UniParser result is written to the correct position in the merged output. This guarantees that the final DataFrame maintains the original log ordering, which is required for correct evaluation against the ground-truth annotations.

As noted in Section 8.2, the pipeline was implemented as a two-stage file-based process rather than an inline streaming system. This decision was driven entirely by compatibility requirements with the LogHub 2.0 evaluation framework, which expects parsers to produce output files in a specific format before evaluation metrics are computed. The file-based design introduces I/O overhead that would not be present in a streaming deployment, and parse time measurements reported in Chapter 10 should be interpreted with this in mind. The logical architecture of the system — route low-confidence lines to a semantic parser, merge results — is independent of this implementation choice and would translate directly to a streaming setting.

9. Experimentation and Testing

9.1. Hardware Setup

The experimental environment used for the DrainUP evaluation differs from the hardware configuration employed by the original LogHub benchmark authors, as summarized in Table *iii*. The LogHub evaluation was conducted on a high-specification server equipped with an Intel Xeon Gold 6226R processor, 256 GB of RAM, and an NVIDIA GeForce RTX 3090 GPU running Ubuntu 16.04. The DrainUP experiments, by contrast, were executed on a consumer-grade laptop with an AMD Ryzen 7 4800H processor, 12 GB of RAM supplemented by 8 GB of swap space, and an NVIDIA GeForce GTX 1060 Ti GPU operating under Ubuntu 24.04 via the Windows Subsystem for Linux 2 (WSL 2).

Table iii: Hardware and software environment comparison between the original LogHub evaluation setup and the DrainUP experimental environment used in this thesis.

Item	LogHub Evaluation Environment	DrainUP Evaluation Environment
CPU	Intel® Xeon® Gold 6226R @ 2.90GHz	AMD Ryzen 7 4800H @ 2.90GHz (16 logical processors)
RAM	256 GB	12 GB + 8 GB Swap
GPU	NVIDIA GeForce RTX 3090	NVIDIA GeForce GTX 1060 Ti
OS	Ubuntu 16.04.7 LTS	Ubuntu 24.04 (WSL 2)

9.2. Benchmark and Dataset Scope

All experiments in this thesis are conducted using the LogHub 2.0 benchmark [37], a widely used evaluation framework in the log parsing literature that provides a curated collection of labeled log datasets extracted from real-world systems. Each dataset consists of raw log lines paired with ground-truth template annotations, enabling objective evaluation of parser output against a known reference.

LogHub 2.0 provides two benchmark configurations that differ in the volume of data evaluated per dataset. The 2k configuration evaluates each parser on a fixed sample of 2,000 log lines per dataset, offering a lightweight and directly comparable evaluation setting that is commonly used in the literature to benchmark parsers under controlled conditions. The full configuration uses the complete annotated set of log lines for each dataset, which vary considerably in size across log sources, and provides a more demanding test of parser robustness and scalability, table *iv* provides the full insights to the actual count of logs per dataset. Both configurations are used in this thesis.

The benchmark encompasses log data from thirteen distinct systems, covering a broad range of software categories and logging styles. These include distributed computing frameworks (HDFS [39], Hadoop [40], Spark [41]), high-performance computing systems (BGL [42], HPC [43]), operating system logs (Linux [44], Mac [45]), networked services and security logs (OpenSSH [46], OpenStack [47], Proxifier [48]), and application-level logs (Apache [49], HealthApp [50], Zookeeper [51]). This diversity is a deliberate property of the benchmark and ensures that parsers are evaluated across substantially different log formats, vocabularies, and structural patterns rather than being optimized for a single logging style.

One dataset available in the LogHub 2.0 benchmark, Thunderbird, was excluded from all experiments in this thesis. Thunderbird is a large-scale supercomputer log dataset whose full corpus is substantially larger than those of the other included datasets. Processing it within the two-stage DrainUP pipeline exceeded the available memory capacity of the hardware used for experimentation (12 GB RAM with 8 GB swap, running Ubuntu 24.04 via WSL 2). Exclusion of Thunderbird was therefore a hardware constraint rather than a methodological choice, and results reported in this thesis should be interpreted accordingly. The remaining thirteen datasets were processed without issue under these hardware conditions.

9.3. Baseline Configuration

To evaluate the results of the hybrid approach, DrainUP is compared against two baseline parsers: unmodified Drain [31] and unmodified UniParser [35]. These two parsers were selected as baselines because they are the direct components of DrainUP, making them the most appropriate points of comparison for this hybrid strategy. Comparing against these baselines allows the results to speak directly to the research questions stated in Chapter 5.

Both baselines are evaluated under identical conditions to DrainUP. They are run on the same thirteen datasets, under both the 2k and full benchmark configurations, and their outputs are evaluated using the same metrics and the same LogHub 2.0 evaluation pipeline and all of them run on the same hardware. No modifications are made to either baseline parser beyond what is strictly necessary to integrate them into the shared evaluation framework.

No additional baselines or third-party parsers are included in the comparison. The scope of the experimental evaluation is kept to the three parsers explained above. This is done in order to maintain a focused analysis of the hybrid mechanism itself rather than a broad survey of the log parsing landscape.

9.4. Evaluation Protocol

Each parser is run once per dataset under each benchmark configuration. No repeated experiments are conducted. The evaluation therefore produces one result per parser per dataset per configuration, with a total of 78 individual runs across the three parsers, thirteen datasets, and two benchmark configurations (2k and full).

For DrainUP, each run follows the four-stage procedure described in section 8 of this thesis. Drain is first executed on the full dataset to produce a structured output with per-line confidence scores. The routing step then partitions the output into high-confidence and low-confidence subsets based on the confidence threshold, and the low-confidence subset is written to the intermediate file consumed by UniParser. UniParser is then executed on this file, and the merge script combines both outputs into the final structured result. This merged output is then passed directly to the LogHub 2.0 evaluation pipeline for metric computation.

For the two baselines, each run consists of a single parser execution on the full dataset followed by direct evaluation of its output. No intermediate filtering or merging steps are applied.

All parsers produce output in the structured format expected by the LogHub 2.0 evaluation framework, which compares each parser's assigned EventId and template strings against the ground-truth annotations provided with each dataset. The framework then computes the full set of evaluation metrics used in this thesis: Grouping Accuracy (GA), Parsing Accuracy (PA), F1 of Grouping Accuracy (FGA), Precision of Template Accuracy (PTA), Recall of Template Accuracy (RTA), F1 of Template Accuracy (FTA), and parse time. These metrics are defined in detail in Section 6.9, and results are reported in Section 10.

Parse time is measured as the time elapsed from the start of parsing to the production of the final structured output, recorded in seconds. For DrainUP, this encompasses only the time spent by the two parsers on actual parsing work. The merge step and intermediate file I/O are explicitly excluded from this measurement, as they are artifacts of the file-based architecture adopted for benchmark compatibility rather than actual costs of the parsing process itself. In a real-world deployment, where the pipeline would operate as an inline streaming system, no such merge step would exist. Reported parse times for DrainUP therefore reflect the combined computational cost of the two parsing stages only.

No additional preprocessing is applied to any dataset beyond what each parser applies internally as part of its standard operation. Log lines are passed to each parser in their raw form as provided by the LogHub 2.0 benchmark.

10. Results — Analysis and Interpretation

10.1. Experimental Setup Recap

This chapter presents and analyzes the results of the experimental evaluation of DrainUP, the proposed hybrid log parser, in comparison to its two constituent baselines: Drain, representing the statistical parsing paradigm, and UniParser, representing the semantic parsing paradigm. The evaluation is organized around two benchmark configurations derived from the LogHub dataset collection: the full dataset, which constitutes the primary evaluation, and the 2k dataset, which serves as a controlled secondary reference. Both configurations span the same thirteen heterogeneous log sources and are assessed under the same six benchmark metrics established by the LogHub evaluation framework [37].

The full dataset evaluation uses the complete annotated log contents as provided by LogHub 2.0, with dataset sizes ranging from 21,320 log lines in Proxifier [52] to over sixteen million lines in Spark [41], as detailed in Table iv. This configuration is considered the primary evaluation because it reflects conditions closest to real-world production environments, where log parsers must process large, continuously growing streams of log data under operational time constraints. The 2k dataset, by contrast, contains exactly 2,000 representative log lines from each dataset, offering a balanced and computationally flexible benchmark that facilitates direct comparison with published baselines in prior literature.

Table iv: Overview of the thirteen evaluation datasets, showing the number of annotated log lines in the full dataset and the number of distinct ground-truth templates in both the full and 2k benchmark configurations. A dash (—) indicates that the 2k template count was not reported in the LogHub 2.0 documentation for that dataset.

Dataset	Annotated Log Lines (Full)	Templates (Full)	Templates (2k)
Apache	51,977	29	—
BGL	4,631,261	320	120
HDFS	11,167,740	46	14
HealthApp	212,394	156	75
HPC	429,987	74	46
Hadoop	179,993	236	114
Linux	23,921	338	118
Mac	100,314	626	341
OpenSSH	638,946	38	27
OpenStack	207,632	48	43
Proxifier	21,320	118	—
Spark	16,075,117	236	36
Zookeeper	74,273	89	50

Three parsers are evaluated throughout this chapter. Drain [31] is configured with its default fixed-depth parse tree parameters and operates as a purely statistical, heuristic-based online parser with no prior training requirement. UniParser [35] is a deep learning model that encodes token and contextual information to assign template labels; it requires a training phase on a representative sample of the target log source before inference. DrainUP implements the hybrid strategy described in Chapter 8: Drain processes each incoming log line and produces a confidence estimate based on its internal tree traversal; lines that exceed a predefined confidence threshold are accepted as parsed, while those falling below the threshold are forwarded to UniParser for a second-pass semantic analysis. This design is intended to preserve Drain's speed advantage on well-structured, high-frequency log patterns while leveraging UniParser's precision for ambiguous or irregular log lines that Drain cannot resolve with confidence.

10.2. Full Dataset Results

The full dataset evaluation constitutes the primary experimental subject of this thesis. Unlike the controlled 2k benchmark, which is built on top of a uniform sample of 2,000 lines from each log source, the full dataset exposes each parser to the complete annotated logs as collected in production environments, with dataset sizes spanning from 21,320 lines in Proxifier to over sixteen million lines in Spark. This configuration is therefore the most relevant for assessing whether the proposed hybrid approach generalizes and scales under conditions that resemble actual site reliability engineering deployments. The following subsections examine each metric in turn, proceeding from grouping-level accuracy metrics to template reconstruction metrics and concluding with parse time, before synthesizing all dimensions into an overall assessment in Section 10.2.5.

10.2.1. Grouping Accuracy (GA)

Grouping Accuracy (as seen in section 6.9.) measures the proportion of log lines that are assigned to the correct template group, weighted by the number of lines belonging to each group. Because this weighting scheme gives disproportionate influence to high-frequency templates, GA tends to reward parsers that correctly handle the most common log patterns even if they fail entirely on rare ones. Despite this limitation, GA remains a standard and an informative indicator of parsing quality, and its results across the thirteen full datasets reveal a clear and consistent pattern: statistical parsers maintain strong performance at scale, while the semantic parser UniParser degrades significantly on the majority of datasets when exposed to the full log contents.

In the following heatmap Figure ix the complete GA results for Drain, DrainUP, and UniParser across all thirteen datasets are presented. The most immediately striking observation is the extent to which UniParser's performance deteriorates relative to what is typically observed on smaller benchmarks. On Apache, both Drain and DrainUP achieve a perfect score of 1.000, whereas UniParser scores only 0.544 — a gap of 0.456. A similar pattern emerges on BGL, where Drain and DrainUP score 0.919 and 0.923 respectively, compared to UniParser's 0.483. On HealthApp, Drain and DrainUP both achieve 0.862 while UniParser falls to 0.446, and on Linux, DrainUP scores 0.778 versus UniParser's 0.268. These results indicate that UniParser's token-level deep learning model, while effective on small curated samples, loses global grouping consistency when confronted with the full diversity and volume of a large log corpus. The fixed-depth parse tree structure

underlying Drain, by contrast, is specifically designed for online, streaming log parsing and maintains its grouping integrity regardless of dataset size.

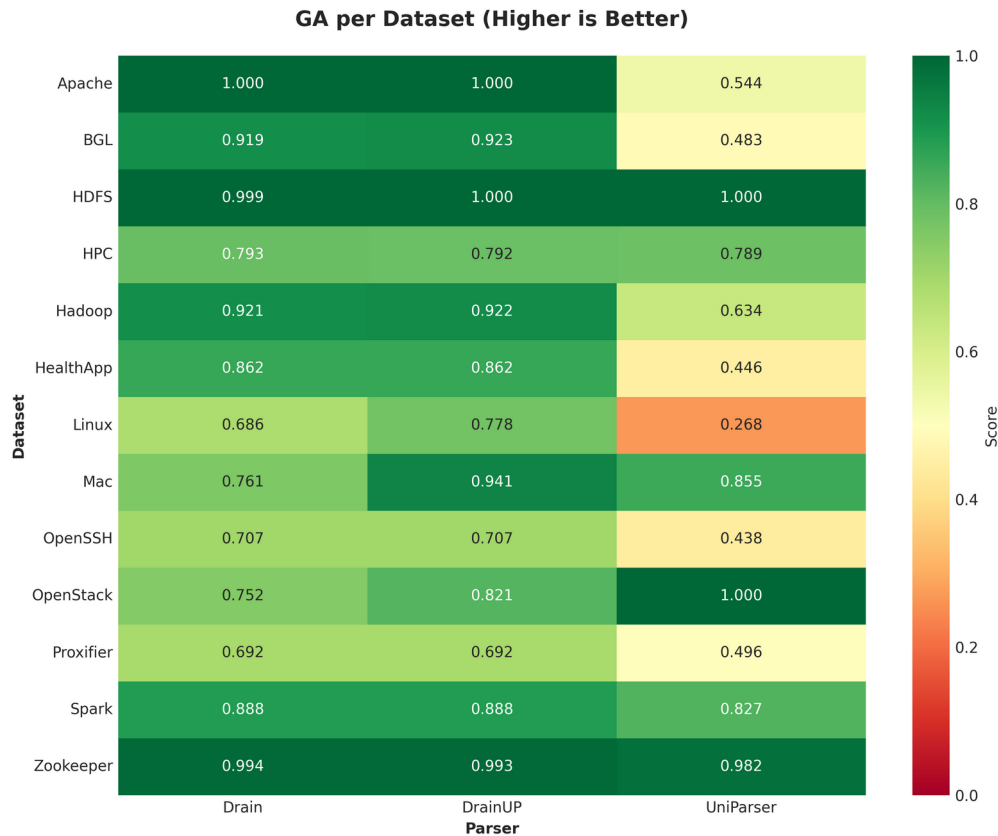


Figure ix: Grouping Accuracy (GA) on the full dataset for all three parsers across the thirteen evaluation datasets.

DrainUP demonstrates consistent improvement over Drain on several datasets where the hybrid second pass meaningfully redirects low-confidence lines to UniParser. The most pronounced gain is observed on Mac, where DrainUP achieves a GA of 0.941 compared to Drain's 0.761, a difference of 0.180. On Linux, DrainUP scores 0.778 against Drain's 0.686, an improvement of 0.092. On OpenStack, DrainUP reaches 0.821 versus Drain's 0.752. These improvements reflect datasets with high template diversity and irregular log patterns, where Drain's fixed-depth tree frequently routes messages to incorrect groups with low internal confidence, triggering the UniParser second pass. In contrast, on datasets with highly repetitive and well-structured log patterns — such as Apache, HDFS, Hadoop, OpenSSH, Proxifier, Spark, and Zookeeper — DrainUP produces results identical or nearly identical to Drain, indicating that the confidence threshold correctly identifies these lines as high-confidence Drain matches and avoids unnecessary invocation of UniParser.

The average GA across all thirteen datasets further illustrates this picture. Drain achieves an average of 0.844, DrainUP achieves 0.871, and UniParser achieves only 0.674. The 0.197-point gap between DrainUP and UniParser on average GA represents a substantial and consistent advantage of the hybrid approach over the semantic baseline at full scale. It is worth noting that UniParser does outperform both Drain and DrainUP on two datasets: OpenStack, where it achieves a perfect 1.000 compared to Drain's 0.752 and DrainUP's 0.821, and HDFS, where all three parsers score at or very close to 1.000. The OpenStack result is particularly notable and suggests that for certain well-defined log formats with a small and stable template vocabulary — OpenStack has only 48 ground-truth templates — UniParser's semantic model can generalize effectively even at scale. These exceptions, however, do not offset the more general trend of UniParser under-performance across the majority of the evaluation tests.

The underlying reason for UniParser's degradation on scale for the GA metric is natural to the fundamental difference between how semantic and statistical parsers handle global grouping. UniParser processes each log line independently, assigning template labels based on token-level learned representations in training stages, but it does not maintain any persistent state about the distribution of current observed templates across the logs. As a result, when the volume and variety of incoming log lines grows, the model's local token-level predictions do not necessarily produce globally consistent groupings. Drain, by contrast, routes every incoming message through a shared fixed-depth parse tree that is updated incrementally, making sure that all lines sharing the same structural prefix are consistently assigned to the same node on the fix-depth tree. This difference makes Drain naturally more robust to scale on the GA metric, and it is precisely this robustness that DrainUP takes into advantage for the majority of log lines it processes. As described in Chapter 8.1., only the lines for which Drain assigns a low internal confidence score are forwarded to UniParser, limiting UniParser's exposure to the cases where it is most likely to help rather than harm grouping consistency.

10.2.2. F1 of Group Accuracy (FGA)

While Grouping Accuracy provides a useful summary of overall parsing correctness, this metric gives a high score to parsers that handle only the most frequent templates correctly, while completely failing on rare but important event types such as error and critical level logs that can be rare but far more important than info logs that are far more common. The F1 of Group Accuracy addresses this limitation by computing, for each ground-truth template, the F1 score of the set of log lines that the parser assigns to the correct group, and then averaging these template-level scores uniformly across all templates regardless of their frequency. This fairness criterion makes FGA a more demanding metric than GA, and it is on this metric that the full-dataset results most clearly reveal the limitations of both baselines and the robustness advantage of the hybrid approach.

The following heatmap Figure x. presents the complete FGA results across all thirteen datasets. The most consequential finding is the presence of catastrophic failures in both Drain and UniParser — scores so low that they indicate near-total inability to correctly group messages for specific template types. Drain collapses to an FGA of 0.010 on HealthApp and 0.007 on OpenStack. These results expose a fundamental limitation of purely heuristic statistical parsers: their grouping behavior is sensitive to the choice of structural parameters, and a configuration that works well on one dataset can fail catastrophically on another.



Figure x: F1 of Group Accuracy (FGA) on the full dataset

UniParser exhibits an equally serious but differently distributed set of failures compared to Drain. On Spark, UniParser achieves an FGA of only 0.001, despite the Spark dataset containing 236 ground-truth templates. Inspection of the identified template count reveals that UniParser produces 400,621 distinct template groups on the full Spark contents, compared to the 236 ground-truth templates and the 259 groups produced by Drain. This extreme over-segmentation indicates that UniParser's token-level model, when applied to the full sixteen-million-line Spark dataset, fragments log messages into an enormous number of spurious micro-templates, each containing only a handful of lines.

DrainUP avoids both categories of failure. It achieves no FGA score below 0.389 across all thirteen datasets — a floor that represents a minimum level of functional grouping accuracy across the entire evaluation tests. On HealthApp, where Drain scores 0.010, DrainUP recovers to 0.450 by routing the low-confidence lines that Drain would have incorrectly merged to UniParser, which is better equipped to distinguish among template types with similar structure. On OpenStack, where Drain scores 0.007, DrainUP achieves 0.697.

The improvement of DrainUP over Drain represents a gain of 0.185 FGA points on average, while the improvement over UniParser is 0.254 points. These are not marginal differences: they reflect a fundamentally more reliable parsing behavior across a heterogeneous collection of log sources. Notably, DrainUP leads or ties for the best FGA score on nine of the thirteen datasets, with the remaining four going to either Drain (Apache, OpenSSH) or UniParser (HDFS, OpenStack). FGA

captures this risk directly by ensuring that each template type, regardless of how rarely it appears in the tests, contributes equally to the overall evaluation score. From this perspective, DrainUP's FGA advantage over both baselines is not merely a statistical improvement: it represents a meaningful reduction in the risk of muting and overlooking real failures on the log event types that operational teams are most likely to care about.

10.2.3. Parsing Accuracy (PA)

Parsing Accuracy is the most strict of all the metrics that we are taking into account from the benchmark: it counts a log line as correctly parsed only when the extracted template and the identified parameters simultaneously match the ground-truth annotation exactly. Even a single error on this extraction is sufficient to cause a complete mismatch for that line. This all-or-nothing requirement makes PA highly sensitive to the precision of token-level labeling, and it consequently favors parsers whose architecture is explicitly designed and trained to optimize this kind of local classification.

The next heatmap Figure *xi* presents the full PA results across the thirteen datasets. UniParser leads on eleven of the thirteen datasets and achieves a substantially higher average PA of 0.745, compared to 0.487 for Drain and 0.433 for DrainUP. This advantage is consistent and expected: UniParser is a deep learning model fine-tuned on the task of assigning correct templates and extracting the correct parameters, and PA is precisely the metric that rewards this kind of training.

The model has learned, from labeled examples, where template boundaries tend to fall for each log source, and it applies this knowledge at inference time to produce annotations that closely match the ground truth. Drain and DrainUP, by contrast, create their template representations from heuristics, in these specifics cases the fixed-depth tree routes messages to groups based on token length and common patterns, without any direct optimization for actual semantics. The result is that Drain's templates tend to be more general.

It is important to interpret UniParser's PA advantage in the context of the broader evaluation rather than in isolation. The same nature that enables UniParser to achieve high PA, is also the root cause of its catastrophic FGA failures on Spark and OpenSSH documented in Section 10.2.2. A model that processes each log line independently and optimizes for token extraction accuracy will tend to produce precise template boundaries on the lines it handles correctly, but it cannot guarantee that the resulting groups are globally consistent across the tests. PA does not penalize this inconsistency: it measures per-line correctness in isolation, without regard to whether lines that share the same ground-truth template are assigned to the same parser output group.

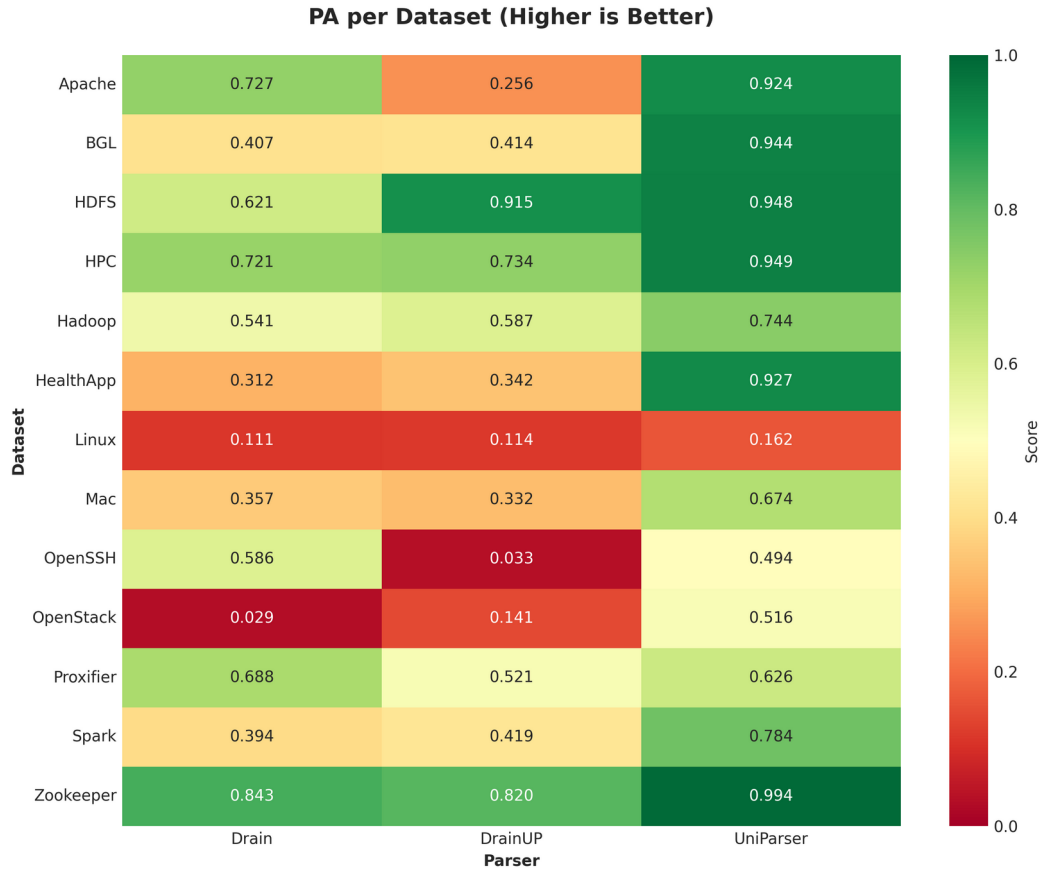


Figure xi: Parsing Accuracy (PA) on the full dataset

Despite DrainUP's lower average PA, there are several datasets on which the hybrid second pass produces meaningful improvements over Drain alone. The most substantial gain is observed on HDFS, where DrainUP achieves a PA of 0.915 compared to Drain's 0.621 — an absolute improvement of 0.294, representing a 47.3% relative increase. But as there are examples in which DrainUP is better than Drain, there are others in which the complete opposite happens. The most notable regression is on Apache, where Drain achieves 0.727 but DrainUP scores only 0.256. This occurs because the Apache log set, despite containing only 29 ground-truth templates, has a proportion of lines that Drain routes with low confidence owing to structural variability in the log format, and these lines are forwarded to UniParser. On OpenSSH, a similar pattern emerges: Drain achieves 0.586 while DrainUP falls to 0.033.

The overall PA picture for the full dataset therefore presents a subtle finding: UniParser leads on the metric it was specifically designed to optimize, but its advantage is achieved at the cost of the global grouping consistency that makes FGA and GA meaningful in practice. DrainUP's lower average PA reflects a deliberate trade-off decision— it sacrifices some token-level boundary precision in exchange for the robustness and speed properties that the other metrics capture — and this trade-off is consistent with the objectives that we were planning from the beginning on the investigation objectives.

10.2.4. Speed at Scale

Beyond accuracy, the computational cost of parsing is a critical aspect that we need to keep in mind in production log analysis environments. Site reliability engineering workflows frequently require log parsers to operate with low latency in order to support near-real-time anomaly detection, alerting, and incident triage. A parser that achieves high accuracy but requires hours of processing time per dataset would simply not be feasible to use and deploy in this context, regardless of its metric scores. The parse time results on the full dataset reveal a dramatic and practically decisive advantage for Drain and DrainUP over UniParser, one that becomes more evident as dataset size increases.

In the following heatmap Figure *xii*, we are presented with the complete parse time measurements in seconds for all three parsers across the thirteen full datasets. As noted in Section 9.1, the absolute times reported here are specific to the DrainUP experimental environment and should be interpreted as relative indicators of computational cost rather than as universal benchmarks. The ratios between parsers, however, are hardware-independent and provide a reliable basis for comparison.



Figure *xii*: Parsing Times for the 3 parsers on the 13 datasets

The most striking observation is the scale of UniParser's parse time on the two largest datasets. On Spark, which contains over sixteen million log lines, UniParser requires 23,205.319 seconds — approximately 6.4 hours — to complete parsing. On HDFS, which contains over eleven million lines, UniParser requires 11,430.718 seconds — approximately 3.2 hours. Drain processes the same Spark corpus in 2,415.335 seconds (approximately 40 minutes), and DrainUP requires essentially identical time at 2,416.067 seconds — an overhead of less than one second across sixteen million lines. On HDFS, Drain

completes in 1,401.211 seconds and DrainUP in 1,401.309 seconds — a difference of 0.098 seconds across eleven million log lines. These figures demonstrate that for datasets dominated by high-confidence Drain matches, the hybrid routing mechanism adds virtually no overhead: UniParser is invoked only for the small fraction of low-confidence lines, and the cost of those additional calls is negligible relative to the total Drain processing time.

It should be noted that the parse time results are partially influenced by the hardware environment described in Section 9.1. UniParser's deep learning inference benefits significantly from state of the art GPUs with fast VRAM bandwidth, and its parse times on higher-specification hardware would be lower in absolute terms. However, two observations limit the extent to which this consideration mitigates the practical concern of actually deploying on a production SRE environment. First, the relative advantage of Drain and DrainUP over UniParser is a natural property that persists across hardware environments: Drain's parse tree traversal is a CPU-bound, memory-efficient operation that scales approximately linearly with the number of input lines, while UniParser's deep learning inference carries a fixed overhead that does not diminish proportionally with hardware improvements. Second, many real-world SRE environments operate on commodity hardware or within containerized cloud environments where large GPU allocations are not always available or cost-effective, making the performance profile observed in the experimental environment representative of a realistic deployment scenario.

10.2.5. Overall Full Dataset Assessment

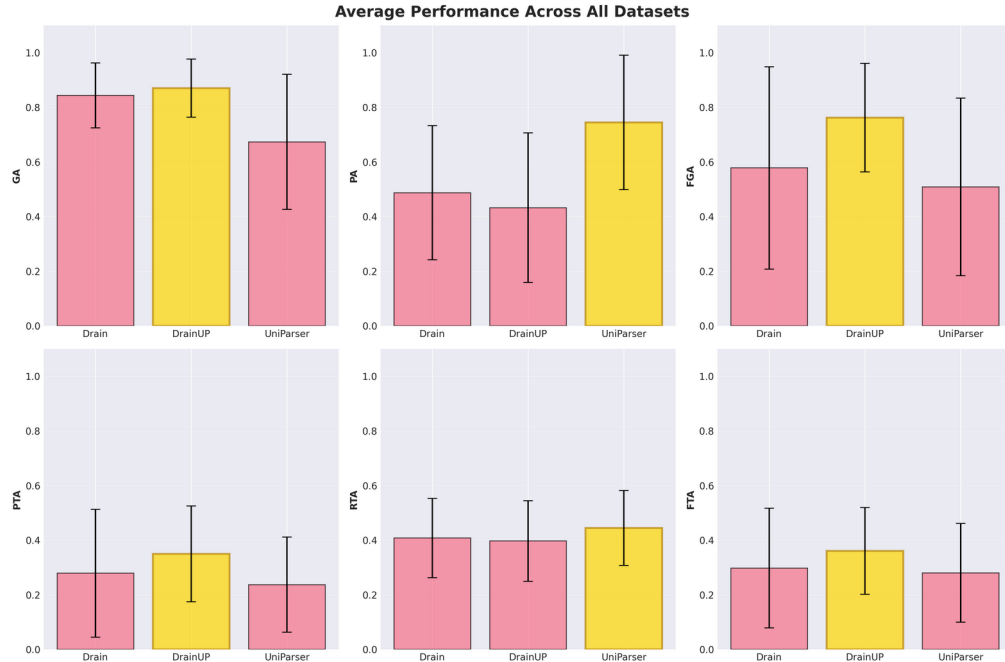


Figure xiii: Average Performance for the 3 parsers across the full dataset

Having examined the full dataset results across all individual metrics, it is possible to synthesize a comprehensive picture of the three parsers' behavior at real production scale. The following Figure xiii provides the summary of the 3 parsers across the 6 different metrics that the benchmark provides.

The results paint a consistent picture. DrainUP leads on four of the seven evaluation dimensions: GA (0.871), FGA (0.763), PTA (0.350), and FTA (0.361). UniParser leads on two: PA (0.745) and RTA (0.445). Drain leads on parse time (217.3 seconds average) but is closely followed by DrainUP (238.1 seconds), while UniParser's average of 3,010.7 seconds places it in a different operational category entirely. The dimension on which Drain leads over DrainUP — parse time — represents a gap of 20.8 seconds on average, a difference that is negligible in any deployment context. The dimensions on which DrainUP leads over Drain, by contrast, represent substantive improvements in parsing quality: the FGA gain of 0.185 points corresponds to a 32% relative improvement in template-fair grouping accuracy, and the elimination of catastrophic grouping failures on HealthApp (Drain FGA 0.010) and OpenStack (Drain FGA 0.007) represents a qualitative improvement in reliability.

The only metric on which DrainUP trails a meaningful comparison is PA, where it scores 0.433 against UniParser's 0.745. As discussed in Section 10.2.3, this gap reflects a normal bias in the PA metric toward trained semantic parsers rather than a general weakness of the hybrid approach. When PA and FGA are considered jointly, UniParser's PA advantage is substantially offset by its FGA weakness, particularly on large and diverse tests where both metrics are most informative.

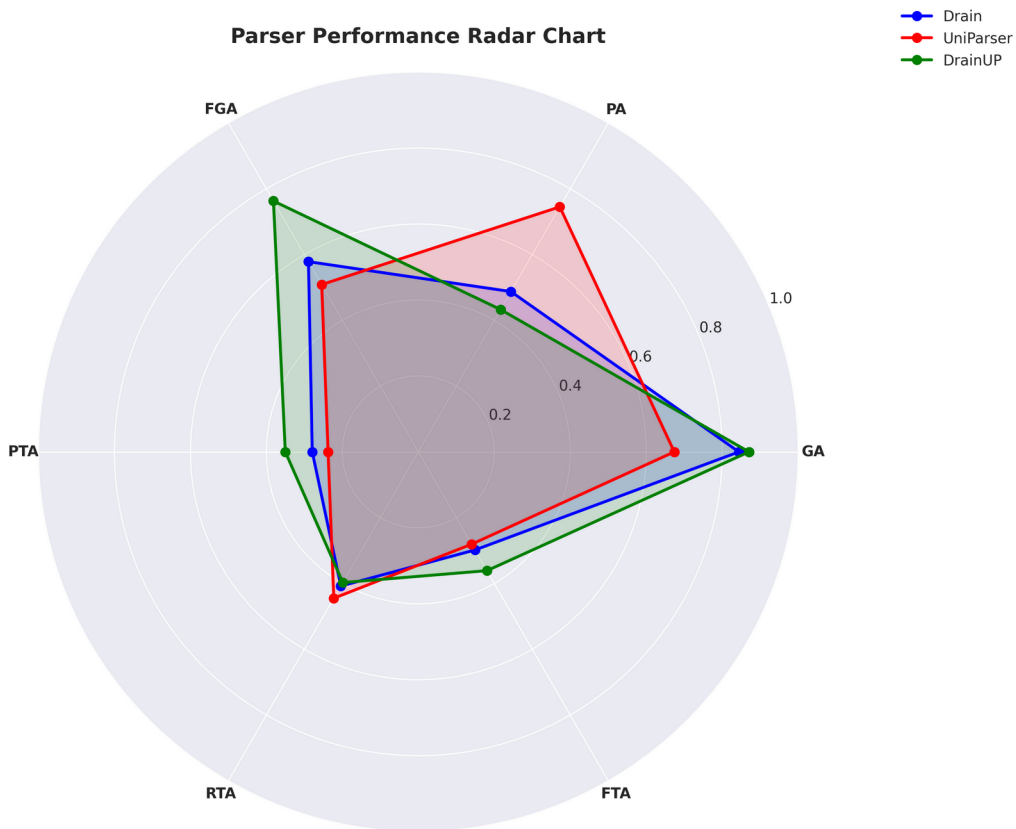


Figure xiv: Parser Performance radar chart for the 6 metrics and the 3 parsers for the full dataset

The results also confirm that the hybrid design achieves its stated objective of combining the complementary strengths of its constituent parsers. On datasets where log messages are highly regular and Drain achieves high internal confidence — Apache, HDFS, Spark, Zookeeper, Hadoop — DrainUP's behavior is essentially identical to Drain's, preserving Drain's speed and grouping accuracy without invoking UniParser unnecessarily. On datasets where log messages are structurally diverse or irregular — Mac, Linux, HPC, HealthApp, OpenStack — the confidence-based routing identifies the cases where Drain is likely to fail, forwards those lines to UniParser, and achieves measurable improvements in GA and FGA relative to Drain alone, at the cost of a moderate increase in parse time. This adaptive behavior is the core mechanism that allows DrainUP to outperform both baselines on average across the full evaluation suite.

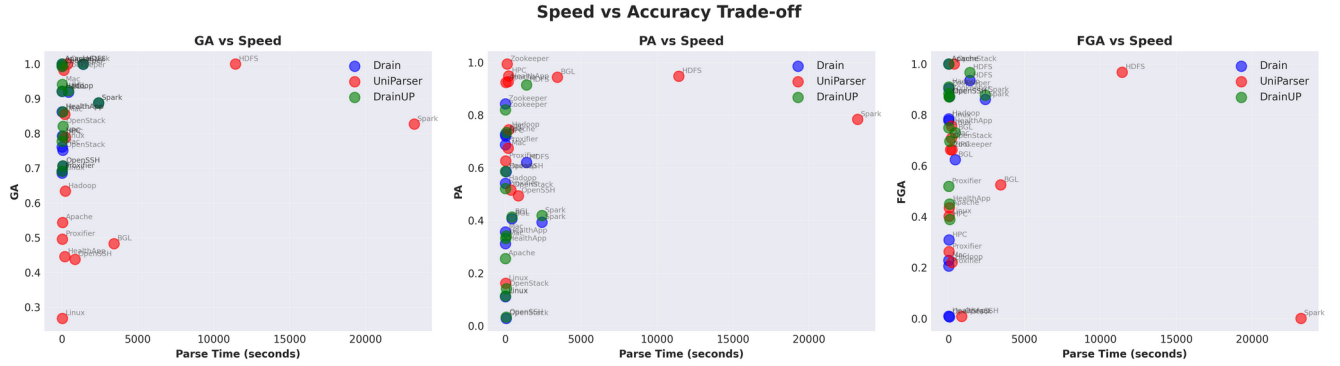


Figure xv: Speed vs Accuracy Trade-off for GA, PA and FGA

10.3. 2k Dataset Results

The 2k dataset evaluation contains exactly 2,000 representative log lines from each of the thirteen log sources and constitutes the secondary evaluation of this thesis. Its primary purpose is to situate the DrainUP results within the established literature: the 2k benchmark is the configuration used by the original LogHub evaluation and by the majority of prior log parsing papers, and results on this benchmark therefore allow direct comparison with published baselines. Because the controlled sample eliminates the scale effects that dominate the full dataset evaluation, the 2k results also serve as a useful diagnostic lens to check in reality which parsers degrade with scale.

10.3.1. Grouping Accuracy (GA) and FGA

The heatmap Figure xvi presents the GA results on the 2k benchmark. The most immediate observation is how much more competitive all three parsers are at this scale relative to the full dataset. The average GA scores are 0.845 for Drain, 0.845 for DrainUP, and 0.788 for UniParser — a considerably narrower spread than the 0.844, 0.871, and 0.674 averages observed on the full dataset. UniParser's GA degradation, which was the most notorious finding of Section 10.2.1, is substantially diminished on the 2k benchmark: the average gap between DrainUP and UniParser narrows from 0.197 to 0.057. This confirms that UniParser's grouping accuracy loss is dependent to scale — the model's stateless token-level inference can maintain reasonable global consistency when exposed to only 2,000 lines per dataset, but loses become notorious as the volume grows to millions of log lines.

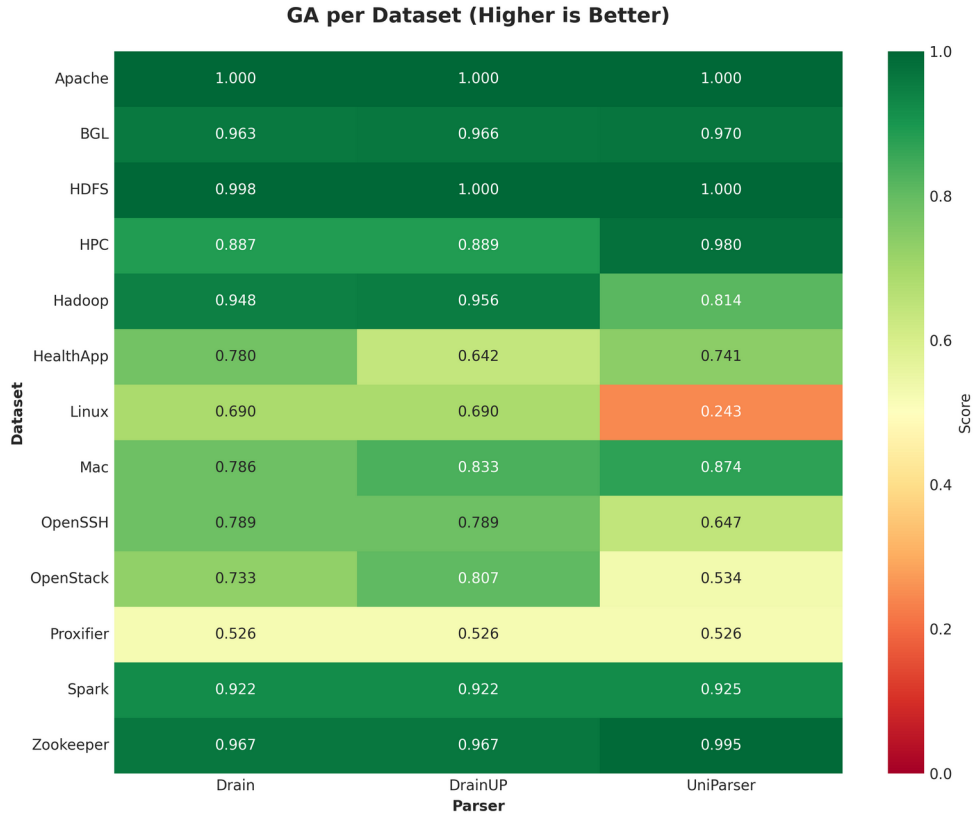


Figure xvi: Grouping Accuracy (GA) on the 2k dataset

The FGA results on the 2k benchmark, presented in Figure xvi, show a more subtle picture that reinforces the central finding from the full dataset. DrainUP achieves the highest average FGA of 0.849, compared to 0.785 for UniParser and 0.739 for Drain. Critically, the catastrophic failure modes that defined both baselines on the full dataset are also present on the controlled 2k sample. Drain collapses to an FGA of 0.117 on OpenStack and 0.351 on HealthApp, the same two datasets where it failed catastrophically on the full dataset, confirming that these failures are natural properties of Drain's parse tree configuration on these specific log formats rather than a scaling issue.



Figure xvii: *F1 of Grouping Accuracy (FGA) on the 2k dataset*

Together, the 2k GA and FGA results confirm two of the key conclusions drawn from the full dataset analysis. First, the advantage of DrainUP over both baselines, which first is its ability to avoid catastrophic FGA failures on any dataset, which we confirmed as well that is not a scale-dependent property but an intrinsic feature of the hybrid design, observable even on small controlled benchmarks. Second, UniParser's GA competitiveness on the 2k benchmark is a controlled-sample result that does not generalize to real-world deployment conditions, where the full dataset results are the more representative evaluation.

10.3.2. Parsing Accuracy (PA)

The heatmap Figure xviii presents the PA results on the 2k benchmark. UniParser maintains its lead with an average PA of 0.745 — identical to its full dataset average — while Drain scores 0.332 and DrainUP scores 0.383. The metric-level argument developed in Section 10.2.3 applies here without qualification: PA inherently favors trained semantic models, and the 2k results reproduce this bias as consistently as the full dataset results.

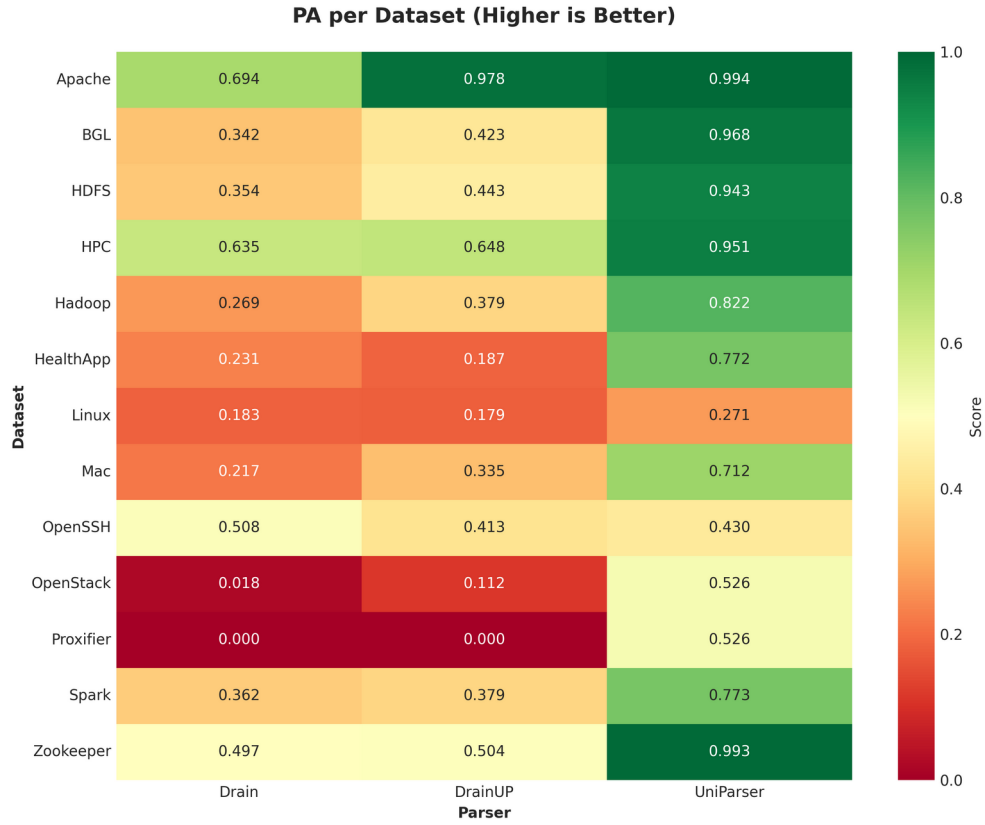


Figure xviii: Parsing Accuracy (PA) on the 2k dataset

10.3.3. Speed on 2k

Then we have the next heatmap Figure *xix* which represent the parsing times for the 2k dataset. At this scale, all three parsers complete each dataset in under five seconds, and the practical distinction between them is negligible from an operational standpoint. Drain averages 0.252 seconds per dataset, DrainUP averages 0.600 seconds, and UniParser averages 2.897 seconds. The relative ordering is identical to the full dataset — Drain is fastest, DrainUP intermediate, UniParser slowest — but the absolute differences are trivial: even UniParser's worst-case time of 4.656 seconds on Mac is well within any reasonable operational window for a 2,000-line sample.

When comparing 2k's results with the full dataset we can see that Drain's parse tree traversal scales linearly with line count, while UniParser's deep learning inference carries a fixed overhead that becomes proportionally more costly as the number of log lines grows. The 2k benchmark therefore provides an optimistic view on the parse time advantage of DrainUP over UniParser, and the full dataset results reflect the more operationally relevant view that we should take into account

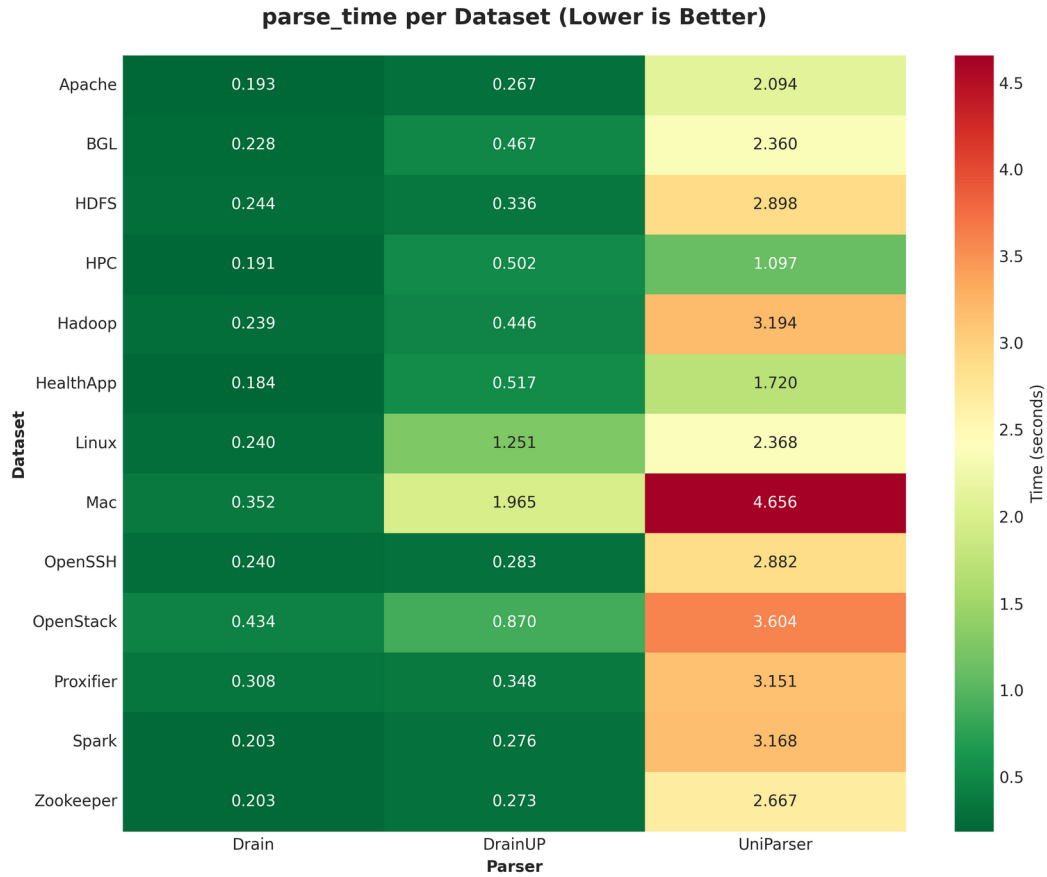


Figure xix: Parsing times on the 2k dataset

10.3.4. Overall 2k Assessment

What the 2k results do confirm without any doubt, is the robustness of DrainUP. The hybrid approach leads on FGA (0.849) even on this small benchmark, ahead of both UniParser (0.785) and Drain (0.739), and it avoids the catastrophic grouping failures that both baselines exhibit on specific datasets. This finding is significant precisely because the 2k benchmark is the configuration used by prior literature: it establishes that DrainUP's robustness advantage is clear under the same controlled conditions in which existing parsers have been evaluated, and is not an artifact of the full dataset scale.

For the purposes of this thesis, the 2k results are therefore presented as a compatibility reference with prior work, while the full dataset evaluation remains the primary basis for all conclusions regarding the practical utility of the hybrid approach.

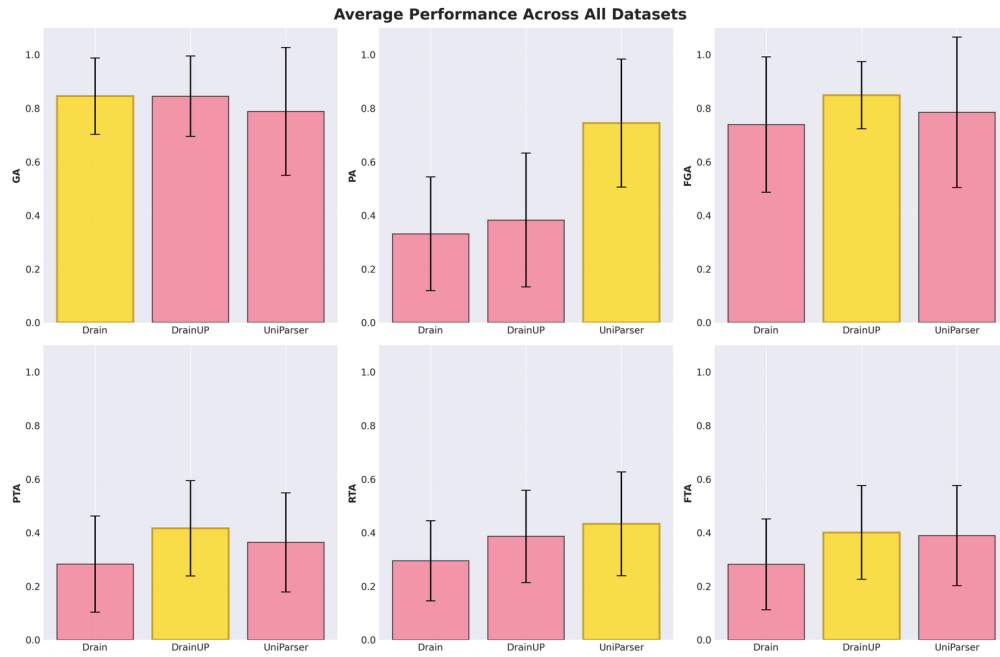


Figure xx: Average Performance for the 3 parsers across the 2k dataset

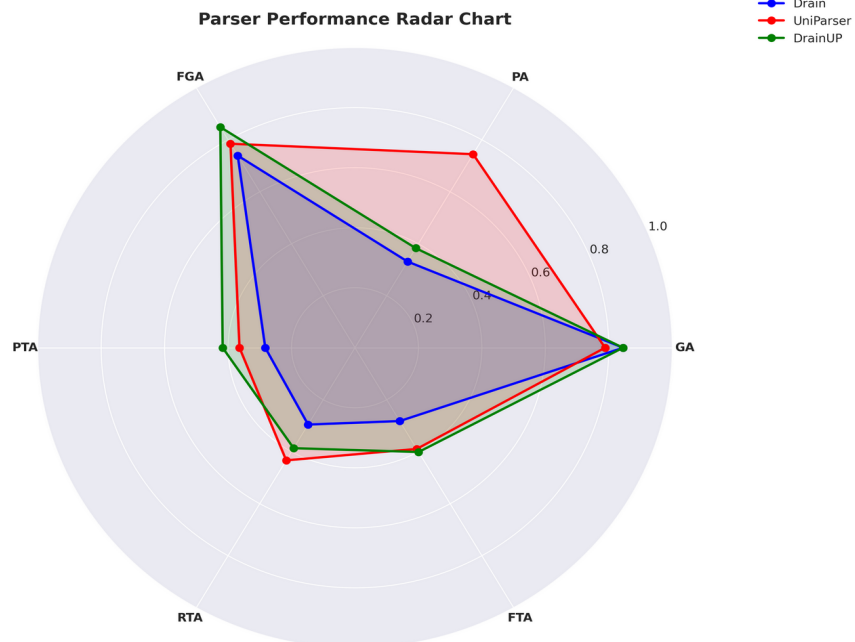


Figure xxi: Parser Performance radar chart for the 6 metrics and the 3 parsers for the 2k dataset

10.4. Cross-Dataset Analysis

In the next section a complementary perspective will be taken, asking which parser wins on each dataset across the three primary metrics (GA, FGA and PA), and what the distribution of wins reveals about the generalization profiles of each approach. In the following table the per-dataset winners on the full dataset for these three metrics is presented.

Table v: Per-dataset metric winners on the full dataset for GA, FGA, and PA. Green shading indicates DrainUP wins; amber shading indicates Drain wins; blue shading indicates UniParser wins; gray indicates a tie. The bottom row summarizes the total win counts per parser per metric.

Dataset	GA Winner	FGA Winner	PA Winner
Apache	Drain / DrainUP	Drain / DrainUP	UniParser
BGL	DrainUP	DrainUP	UniParser
HDFS	DrainUP / UniParser	UniParser	UniParser
HPC	Drain	UniParser	UniParser
Hadoop	DrainUP	DrainUP	UniParser
HealthApp	Drain / DrainUP	UniParser	UniParser
Linux	DrainUP	Drain	UniParser
Mac	DrainUP	DrainUP	UniParser
OpenSSH	Drain / DrainUP	Drain / DrainUP	Drain
OpenStack	UniParser	UniParser	UniParser
Proxifier	Drain / DrainUP	DrainUP	Drain
Spark	Drain / DrainUP	DrainUP	UniParser
Zookeeper	Drain	Drain	UniParser
Wins	Drain 2 · DrainUP 4 · Uni 1 · Tie 6	Drain 2 · DrainUP 5 · Uni 4 · Tie 2	Drain 2 · DrainUP 0 · Uni 11 · Tie 0

Several observations emerge from this dataset-level breakdown that are not fully visible in the average metric scores alone. On GA, DrainUP leads or ties on ten of the thirteen datasets, with UniParser leading on OpenStack (1.000 versus DrainUP's 0.821) and both parsers tying on HDFS at near perfect scores. The concentration of UniParser's GA wins on datasets with small, well-defined template vocabularies (like OpenStack) reinforces the conclusion that UniParser's semantic model can generalize effectively at scale under a narrow and constrained set of conditions, but fails to maintain grouping consistency when log diversity and volume increase.

On FGA, the win distribution provide even more insights. DrainUP leads on seven datasets outright and ties on two (Apache with Drain, OpenSSH with Drain), for a total of nine datasets on which DrainUP achieves the highest or joint-highest FGA. Drain leads on two datasets only. UniParser leads on only two datasets: HealthApp (0.756) and OpenStack (1.000), both

cases where its semantic model happens to segment the log lines in a way that aligns well with the ground-truth template structure. The catastrophic FGA failures of both Drain (HealthApp 0.010, OpenStack 0.007) and UniParser (Spark 0.001, OpenSSH 0.009) appear directly in the win table as absence of wins on those datasets, making visible at a glance the risk that either baseline poses as a general-purpose parser.

On PA, UniParser leads on eleven of the thirteen datasets, with Drain leading on Proxifier (0.688) and both Drain and DrainUP scoring identically on Linux (0.111 and 0.114 respectively, within rounding). This near-universal PA dominance by UniParser confirms the metric-level conclusion reached in Section 10.2.3: PA is structurally biased toward the semantic parsing paradigm, and the win count reflects this bias rather than a general superiority of UniParser on overall parsing capabilities.

Three datasets stand out as universally difficult across all parsers and both metrics: Linux, Proxifier, and OpenStack. Linux presents challenges for all parsers on PA due to its high template count (338) and irregular, human-authored log format, as discussed in Section 10.2.3, and no parser achieves a GA above 0.778 on the full set of logs. Proxifier scores are uniformly low on FGA across all parsers (DrainUP 0.519, Drain 0.206, UniParser 0.263), reflecting a log format whose structural variability exceeds what any of the evaluated approaches can reliably handle. OpenStack exhibits the largest cross-parser variance: UniParser achieves perfect GA and FGA scores while Drain collapses to near-zero FGA, with DrainUP occupying an intermediate position. These hard datasets serve as natural stress tests that reveal the limits of current log parsing techniques and suggest directions for future work on parser robustness.

10.5. Connecting Results to Research Questions

This section revisits the three research questions stated in section 5 and answers each explicitly on the results presented based on the experimental evidence that can be found on previous sections.

10.5.1. RQ1: Does DrainUP achieve superior accuracy relative to its standalone constituent parsers?

The experimental evidence provides a clear and affirmative answer to this question, qualified by the metrics discussed throughout section 5 of this thesis. On the two most informative accuracy metrics GA and FGA, DrainUP outperforms both Drain and UniParser on average across all thirteen full set of logs. Its average GA of 0.871 exceeds Drain's 0.844 by 0.027 points and UniParser's 0.674 by 0.197 points, a substantial advantage at real scale. Its average FGA of 0.763 exceeds Drain's 0.578 by 0.185 points and UniParser's 0.509 by 0.254 points, with the additional and practically significant property that it avoids catastrophic failures on any dataset. On PA, DrainUP scores 0.433 against UniParser's 0.745, a gap that reflects the structural bias of the PA metric toward trained semantic models rather than a fundamental weakness of the hybrid approach. The answer to RQ1 is therefore: yes, DrainUP achieves superior accuracy on the metrics that best reflect real-world parsing utility, at the cost of lower PA on a metric that inherently rewards the design choices of its semantic constituent.

10.5.2. RQ2: How does the confidence-based routing mechanism affect the accuracy–speed trade-off?

The routing mechanism is the central architectural contribution of DrainUP, and the results reveal that it achieves its intended purpose with a favorable trade-off profile. On the speed dimension, DrainUP adds an average of 20.8 seconds (which is approximately a 9.6% difference) to Drain's parse time across the thirteen full datasets. On datasets where the log format is highly regular and Drain's confidence threshold routes nearly all lines through the fast statistical path, on Spark and HDFS, the two largest datasets, DrainUP's parse time differs from Drain's by less than one second. In every case, the routing mechanism correctly identifies the datasets where the semantic second pass is most needed and limits UniParser's involvement to the fraction of lines where it is most likely to be beneficial.

On the accuracy side, the routing mechanism produces its most compelling gain on the datasets where it is most active. The HDFS PA improvement of 0.294 points (+47.3% relative to Drain) is directly attributable to the second pass routing ambiguous log lines. The GA improvements on Mac (+0.180), Linux (+0.092), and OpenStack (+0.069) reflect datasets where Drain's fixed-depth tree produces systematically incorrect groupings for a subset of templates, and the confidence threshold correctly identifies these as low-confidence cases. The answer to RQ2 is therefore: the routing mechanism achieves a highly favorable trade-off, adding minimal computational overhead on most datasets while delivering targeted accuracy improvements on the datasets where heuristic parsing is most likely to fail.

10.5.3. RQ3: What deployment considerations and configuration guidelines can be derived from the benchmark results?

The benchmark results already provide a strong empirical foundation from which practical deployment guidelines and recommendations can be made. Cross-dataset generalization is arguably the most operationally important aspect for a log parser deployed in production environments with multiple log sources, and the results reveal consistent patterns that directly inform configuration decisions. DrainUP's FGA scores range from a minimum of 0.389 (HPC) to a maximum of 1.000 (Apache and HDFS), a range that reflects natural variation in dataset difficulty rather than catastrophic failure on any specific log type. By contrast, Drain's FGA scores range from 0.007 (OpenStack) to 1.000, and UniParser's from 0.001 (Spark) to 1.000, both exhibiting near-zero scores on at least two datasets, this indicates that each baseline has systematic blind spots that an engineer deploying the parser without prior knowledge of the target log format could not anticipate or mitigate. These failure patterns, combined with the routing activity and speed data discussed in RQ2, are the primary basis for the deployment guidelines developed in the following section 10.6. The answer to RQ3 is therefore: the benchmark results reveal consistent and actionable patterns in workload distribution, and failure modes that can be translated into concrete deployment recommendations for SRE teams considering adoption of the hybrid approach or simply a new study or thesis trying to continue the work already exposed in this thesis.

10.6. Deployment Guidelines and Practical Recommendations

The experimental results presented in this section, taken together, form a sufficient empirical basis from which a set of practical recommendations can be created for teams considering the adoption of DrainUP or a similar hybrid parsing architecture in production SRE environments. These guidelines are not steps derived from an actual live production deployment, but informed recommendations based in the behavior observed across the thirteen heterogeneous datasets under the two benchmark configurations from LogHub [37]. These recommendations are organized around three practical concerns: architecture adaptation, dataset-driven parser selection, and computational planning.

On architecture adaptation for real-time environments. The DrainUP pipeline evaluated in this thesis operates as a two-stage, file-based process. As specified in Section 8.2, this design was chosen entirely by compatibility requirements with the LogHub 2.0 evaluation framework and is not a fundamental architectural constraint. In a production deployment, the same logical pipeline translates naturally to a streaming architecture: each incoming log line would be routed inline to either the statistical or semantic component based on the confidence score produced at parse time, eliminating the intermediate file writes and the merge step entirely. The routing mechanism itself (which is basically a single threshold comparison on a numerical confidence score) is lightweight enough to operate without measurable latency overhead, as evidenced by the parse time difference between Drain and DrainUP on the largest datasets evaluated. Teams implementing this system in a real-time context should therefore treat the file-based pipeline as a benchmark artifact and prioritize an inline routing design from the outset.

On log source characteristics and parser selection. The cross-dataset results in Section 10.4 reveal that DrainUP's advantage over both baselines is not uniformly distributed, but it is most pronounced on log sources with high template diversity, irregular structure, or a mix of high-frequency routine messages and low-frequency anomalous events. The datasets on which DrainUP delivers the largest FGA improvements over Drain (HealthApp, OpenStack, Mac, Linux) share these characteristics. Conversely, on log sources that are highly structured and dominated by a small number of frequently recurring templates (Apache, HDFS, Hadoop) DrainUP's behavior converges to Drain's, and the hybrid overhead is effectively zero. This pattern provides actionable guidance for practitioners or teams trying to continue the work exposed in this thesis: log sources that are well-structured and exhibit low template diversity are adequately served by a standalone statistical parser, while log sources with irregular formats, large template vocabularies, or a meaningful proportion of rare event types are the primary candidates for hybrid routing like DrainUP. When onboarding a new log source with no prior knowledge of its structure, the hybrid approach is the safer default, as it avoids the catastrophic FGA failures that both baselines exhibit on specific datasets and that cannot be anticipated without dataset-specific inspection.

On computational planning and hardware constraints. The speed results on the full dataset establish a clear operational boundary. UniParser's deep learning inference, even on modest hardware, requires processing times that are incompatible with what is required when handling with incidents (approximately 6.4 hours on the sixteen-million-line Spark dataset and 3.2 hours on the eleven-million-line HDFS dataset in the experimental environment). DrainUP's parse times on the same datasets differ from Drain's by less than one second, because the confidence threshold correctly limits UniParser's involvement to a small fraction of lines on these large log files. For teams operating under strict latency constraints (which is almost always in production environments), this result suggests that the hybrid approach is feasible even at large scale,

provided that the confidence threshold is configured to keep the UniParser-routed fraction within acceptable bounds. On smaller log sources or in environments where GPU acceleration is available, UniParser's inference times are substantially lower in absolute terms, and the latency concern lowers accordingly. The general principle that results from the speed results is that the computational cost of DrainUP scales primarily with the fraction of lines routed to UniParser, not with the total log volume, making the confidence threshold the primary lever for managing operational cost in production. This confidence threshold can be fine tuned as the teams require since the code is open-source.

11. Conclusions

Log parsing remains one of the most practically important bottlenecks in modern SRE and incident resolution practices. As described in the opening chapters of this thesis, the main problem and issue with it is not the absence of capable parsers but the fundamental burden of having a reliable way of parsing efficiently and effectively millions of logs in a production environment with heterogeneous log sources. Then, as parsers were explored, two types of them became important to the work described in this thesis. Statistical parsers provide fast, globally consistent template grouping but fail systematically on structurally irregular or diverse log formats. Semantic parsers achieve precise per-line token extraction but lose global grouping coherence at scale and carry computational costs that are incompatible with production latency requirements. DrainUP was designed to address this gap directly, not by replacing either paradigm but by combining them through a routing mechanism that allocates parsing responsibility based on evidence of per-line confidence rather than a fixed architectural commitment to either approach. Such approach designed to tackle the modern problems of the log parsing world.

The technical contribution of this thesis is a hybrid log parsing pipeline implemented in Python that modifies Drain's internal matching function to expose a numerical confidence score for every log line it processes, and uses that score to route low-confidence lines to UniParser for a semantic second pass. The modification is minimal and localized, but its effect on parsing behavior as seen in the result section, is substantial. The pipeline was implemented as a two-stage file-based process to maintain compatibility with the LogHub 2.0 evaluation framework, a deliberate engineering trade-off that introduces computing overhead, that will not be present if this new approach was intended to be deployed on a real production environment.

The experimental results provide clear and consistent answers to the three research questions that motivated this work. On accuracy, DrainUP outperforms both Drain and UniParser on the metrics that best reflect real-world parsing utility, GA and FGA, while avoiding the catastrophic grouping failures that both baselines exhibit on specific log sources. The absence of near-zero FGA scores on any evaluated dataset is not a minor improvement; it represents a qualitative shift in reliability that directly reduces the risk of silently muting rare but operationally critical log event types, which are the most important events when triaging a real production incident or outage. On the accuracy-speed aspect of the parser, the confidence-based routing mechanism achieves its intended purpose with great news: DrainUP adds an average of 20.8 seconds to Drain's parse time across the full dataset suite, a difference that is negligible in any operational context, while delivering targeted accuracy improvements precisely on the datasets where heuristic parsing is most likely to fail. The routing mechanism

correctly identifies high-confidence log sources and avoids invoking UniParser without a good reason, preserving Drain's speed advantage where it matters most.

On the practical deployment part, the benchmark results shows consistent and actionable patterns that translate into concrete guidance for teams considering adoption or future work that might use this Thesis as base for future research and development. DrainUP's routing behavior across thirteen datasets reveals that the hybrid approach is most valuable on log sources with high template diversity, irregular structure, or a meaningful proportion of rare event types. On highly regular, high-volume log sources, the hybrid overhead converges to zero and the architecture sticks to Drain's behavior as intended by design. These patterns, developed in detail in Section 10.6, provide a practical framework for configuration decisions, computational planning, and log source characterization.

This thesis has several limitations that future work should address. The confidence threshold of 0.5 was used as a fixed default throughout the evaluation and was not varied experimentally; the relationship between threshold value, UniParser routing fraction, and the resulting accuracy-latency trade-off curve remains an open question and route for future works to follow. UniParser requires a training phase on a representative sample of each target log source, a dependency that limits its functionality to work on any log source as is out of the box. Parse time measurements are hardware-dependent and were collected in a modest environment, meaning that UniParser's absolute times would improve on high-end hardware, though the relative ordering and scaling behavior would remain consistent.

Several paths for future works surface from the work presented in this thesis and from the open questions that the results helped to point out. The most clear one is a more in depth study of confidence threshold behavior, which would involve running the full benchmark evaluation across a range of threshold values, such would produce a great insight that will directly impact and improve the guidance provided in Section 10.6. Beyond threshold tuning, testing DrainUP or an equivalent hybrid architecture against real production log streams would validate whether the generalization properties observed in this controlled evaluation hold under the continuous format evolution and operational noise that characterize live production systems. More ambitiously, the routing mechanism developed here is not limited to only two parsers: a multi-stage pipeline incorporating additional specialized parsers for specific log families, or a more complex routing strategy that selects among more than two candidates based on confidence (or other metrics and mechanisms), represents a natural architectural extension that could push hybrid parsing robustness further. The results of this thesis suggest that the hybrid paradigm of combining complementary parsing strategies through a routing mechanisms rather than committing to a single approach is a productive direction for future log parsing research.

12. References

- [1] Z. M. Jiang, A. E. Hassan, P. Flora, and G. Hamann, “Abstracting Execution Logs to Execution Events for Enterprise Applications (Short Paper),” in *2008 The Eighth International Conference on Quality Software*, Oxford, United Kingdom: IEEE, Aug. 2008, pp. 181–186. doi: 10.1109/QSIC.2008.50.
- [2] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, “Site Reliability Engineering”.
- [3] Datadog, “Grok Parser,” Datadog Infrastructure and Application Monitoring. Accessed: Nov. 30, 2025. [Online]. Available: https://docs.datadoghq.com/service_management/events/pipelines_and_processors/grok_parser/
- [4] W. Dobrowolski, M. Nikodem, and O. Unold, “Software Failure Log Analysis for Engineers—Review,” *Electronics*, vol. 12, no. 10, p. 2260, Jan. 2023, doi: 10.3390/electronics12102260.
- [5] “Log Search,” Sumo Logic Docs. Accessed: Oct. 12, 2025. [Online]. Available: <https://help.sumologic.com/docs/search/>
- [6] G. Xu and A. Rountev, “Precise Memory Leak Detection for Java Software Using Container Profiling”.
- [7] V. Beck, M. Landauer, M. Wurzenberger, F. Skopik, and A. Rauber, “System Log Parsing with Large Language Models: A Review,” May 15, 2025, *arXiv*: arXiv:2504.04877. doi: 10.48550/arXiv.2504.04877.
- [8] Z. Jiang *et al.*, “A Large-Scale Evaluation for Log Parsing Techniques: How Far Are We?,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, Vienna Austria: ACM, Sep. 2024, pp. 223–234. doi: 10.1145/3650212.3652123.
- [9] H. Li, W. Shang, and A. E. Hassan, “Which log level should developers choose for a new logging statement?,” *Empir. Softw. Eng.*, vol. 22, no. 4, pp. 1684–1716, Aug. 2017, doi: 10.1007/s10664-016-9456-2.
- [10] “Logs,” OpenTelemetry. Accessed: Nov. 30, 2025. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/logs/>
- [11] “z/OS.” Accessed: Nov. 30, 2025. [Online]. Available: <https://www.ibm.com/docs/en/zos/2.5.0?topic=wisl-log-stream>
- [12] Z. Li *et al.*, “Did We Miss Something Important? Studying and Exploring Variable-Aware Log Abstraction,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, Melbourne, Australia: IEEE, May 2023, pp. 830–842. doi: 10.1109/ICSE48619.2023.00078.
- [13] “Log Parsing 101 - A Beginner’s Guide to Structured Data,” SigNoz. Accessed: Nov. 30, 2025. [Online]. Available: <https://signoz.io/guides/log-parsing/>
- [14] “Caching Home,” Amazon Web Services, Inc. Accessed: Nov. 30, 2025. [Online]. Available: <https://aws.amazon.com/caching/>
- [15] “Throughput vs Latency - Difference Between Computer Network Performances - AWS,” Amazon Web Services, Inc. Accessed: Nov. 30, 2025. [Online]. Available: <https://aws.amazon.com/compare/the-difference-between-throughput-and-latency/>
- [16] C. C. Editor, “hashing - Glossary | CSRC.” Accessed: Nov. 30, 2025. [Online]. Available: <https://csrc.nist.gov/glossary/term/hashing>
- [17] C. C. Editor, “encode - Glossary | CSRC.” Accessed: Nov. 30, 2025. [Online]. Available: <https://csrc.nist.gov/glossary/term/encode>

- [18] Y.-Q. Zhu, J.-Y. Deng, J.-C. Pu, P. Wang, S. Liang, and W. Wang, “ML-Parser: An Efficient and Accurate Online Log Parser,” *J. Comput. Sci. Technol.*, vol. 37, no. 6, pp. 1412–1426, Dec. 2022, doi: 10.1007/s11390-021-0730-4.
- [19] “Elasticsearch ingest pipelines | Elastic Docs.” Accessed: Nov. 30, 2025. [Online]. Available: <https://www.elastic.co/docs/manage-data/ingest/transform-enrich/ingest-pipelines>
- [20] J. Zhu *et al.*, “Tools and Benchmarks for Automated Log Parsing,” Jan. 03, 2019, *arXiv*: arXiv:1811.03509. doi: 10.48550/arXiv.1811.03509.
- [21] C. C. Editor, “Semantic matching - Glossary | CSRC.” Accessed: Nov. 30, 2025. [Online]. Available: https://csrc.nist.gov/glossary/term/semantic_matching
- [22] “What is Machine Learning? | IBM.” Accessed: Nov. 30, 2025. [Online]. Available: <https://www.ibm.com/think/topics/machine-learning>
- [23] S. Zhang *et al.*, “Instruction Tuning for Large Language Models: A Survey,” Oct. 06, 2025, *arXiv*: arXiv:2308.10792. doi: 10.48550/arXiv.2308.10792.
- [24] H. Dai, H. Li, C.-S. Chen, W. Shang, and T.-H. Chen, “Logram: Efficient Log Parsing Using n-Gram Dictionaries,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 879–892, Mar. 2022, doi: 10.1109/TSE.2020.3007554.
- [25] “logpai/loghub-2.0: A Large-scale Evaluation for Log Parsing Techniques: How Far are We? [ISSTA’24].” Accessed: Oct. 12, 2025. [Online]. Available: <https://github.com/logpai/loghub-2.0>
- [26] BlueLionLogram, *BlueLionLogram/Logram*. (Oct. 14, 2025). Python. Accessed: Oct. 13, 2025. [Online]. Available: <https://github.com/BlueLionLogram/Logram>
- [27] R. Vaarandi and M. Pihelgas, “LogCluster - A data clustering and pattern mining algorithm for event logs,” in *2015 11th International Conference on Network and Service Management (CNSM)*, Barcelona, Spain: IEEE, Nov. 2015, pp. 1–7. doi: 10.1109/CNSM.2015.7367331.
- [28] R. Vaarandi, “SLCT - simple logfile clustering tool,” SLCT - simple logfile clustering tool. Accessed: Oct. 25, 2025. [Online]. Available: <https://ristov.github.io/slct/>
- [29] H. Hamooni, B. Debnath, J. Xu, H. Zhang, G. Jiang, and A. Mueen, “LogMine: Fast Pattern Recognition for Log Analytics,” in *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, Indianapolis Indiana USA: ACM, Oct. 2016, pp. 1573–1582. doi: 10.1145/2983323.2983358.
- [30] K. Shima, “Length Matters: Clustering System Log Messages using Length of Words,” *arXiv.org*. Accessed: Oct. 12, 2025. [Online]. Available: <https://arxiv.org/abs/1611.03213v1>
- [31] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in *2017 IEEE International Conference on Web Services (ICWS)*, Honolulu, HI, USA: IEEE, Jun. 2017, pp. 33–40. doi: 10.1109/ICWS.2017.13.
- [32] Y. Huo, Y. Su, C. Lee, and M. R. Lyu, “SemParser: A Semantic Parser for Log Analytics,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, May 2023, pp. 881–893. doi: 10.1109/ICSE48619.2023.00082.
- [33] Y. Huo, *SemParser*. (Oct. 26, 2025). Python. Accessed: Oct. 25, 2025. [Online]. Available: <https://github.com/YintongHuo/SemParser>

- [34] V.-H. Le and H. Zhang, “Log Parsing with Prompt-based Few-shot Learning,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, May 2023, pp. 2438–2449. doi: 10.1109/ICSE48619.2023.00204.
- [35] Y. Liu *et al.*, “UniParser: A Unified Log Parser for Heterogeneous Log Data,” in *Proceedings of the ACM Web Conference 2022*, in WWW ’22. New York, NY, USA: Association for Computing Machinery, Apr. 2022, pp. 1893–1901. doi: 10.1145/3485447.3511993.
- [36] Curated by LOGPAI, “Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics.” Zenodo, Jul. 31, 2023. doi: 10.5281/ZENODO.1144100.
- [37] “loghub-2.0/benchmark/logparser/logmatch at main · logpai/loghub-2.0,” GitHub. Accessed: Dec. 03, 2025. [Online]. Available: <https://github.com/logpai/loghub-2.0/tree/main/benchmark/logparser/logmatch>
- [38] “DrainUp/benchmark/logparser/Drain/Drain.py at main · YeyoM/DrainUp,” GitHub. Accessed: Mar. 07, 2026. [Online]. Available: <https://github.com/YeyoM/DrainUp/blob/main/benchmark/logparser/Drain/Drain.py>
- [39] “HDFS Architecture Guide.” Accessed: Mar. 08, 2026. [Online]. Available: https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html
- [40] “Apache Hadoop.” Accessed: Mar. 08, 2026. [Online]. Available: <https://hadoop.apache.org/>
- [41] “Apache Spark™ - Unified Engine for large-scale data analytics.” Accessed: Mar. 01, 2026. [Online]. Available: <https://spark.apache.org/>
- [42] “IBM Blue Gene,” *Wikipedia*. Feb. 18, 2026. Accessed: Mar. 08, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=IBM_Blue_Gene&oldid=1339104572
- [43] L. A. N. Laboratory, “HPC Systems,” Los Alamos National Laboratory. Accessed: Mar. 08, 2026. [Online]. Available: <https://www.lanl.gov/engage/organizations/aldsct/hpc/hpc-systems>
- [44] “Public Security Log Sharing.” Accessed: Mar. 08, 2026. [Online]. Available: <https://log-sharing.dreamhosters.com/>
- [45] “macOS,” *Wikipedia*. Mar. 07, 2026. Accessed: Mar. 08, 2026. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=MacOS&oldid=1342225555>
- [46] “OpenSSH.” Accessed: Mar. 08, 2026. [Online]. Available: <https://www.openssh.org/>
- [47] “Open Source Cloud Computing Infrastructure - OpenStack.” Accessed: Mar. 08, 2026. [Online]. Available: <https://www.openstack.org/>
- [48] “Proxifier - The Most Advanced Proxy Client.” Accessed: Mar. 08, 2026. [Online]. Available: <https://www.proxifier.com/>
- [49] “Welcome! - The Apache HTTP Server Project.” Accessed: Mar. 08, 2026. [Online]. Available: <https://httpd.apache.org/>
- [50] “Health App - Apps on Google Play.” Accessed: Mar. 08, 2026. [Online]. Available: <https://play.google.com/store/apps/details?id=com.healthsafety.app&hl=en>
- [51] “Apache ZooKeeper.” Accessed: Mar. 08, 2026. [Online]. Available: <https://zookeeper.apache.org/>
- [52] “Proxifier - The Most Advanced Proxy Client.” Accessed: Mar. 01, 2026. [Online]. Available: <https://www.proxifier.com/>